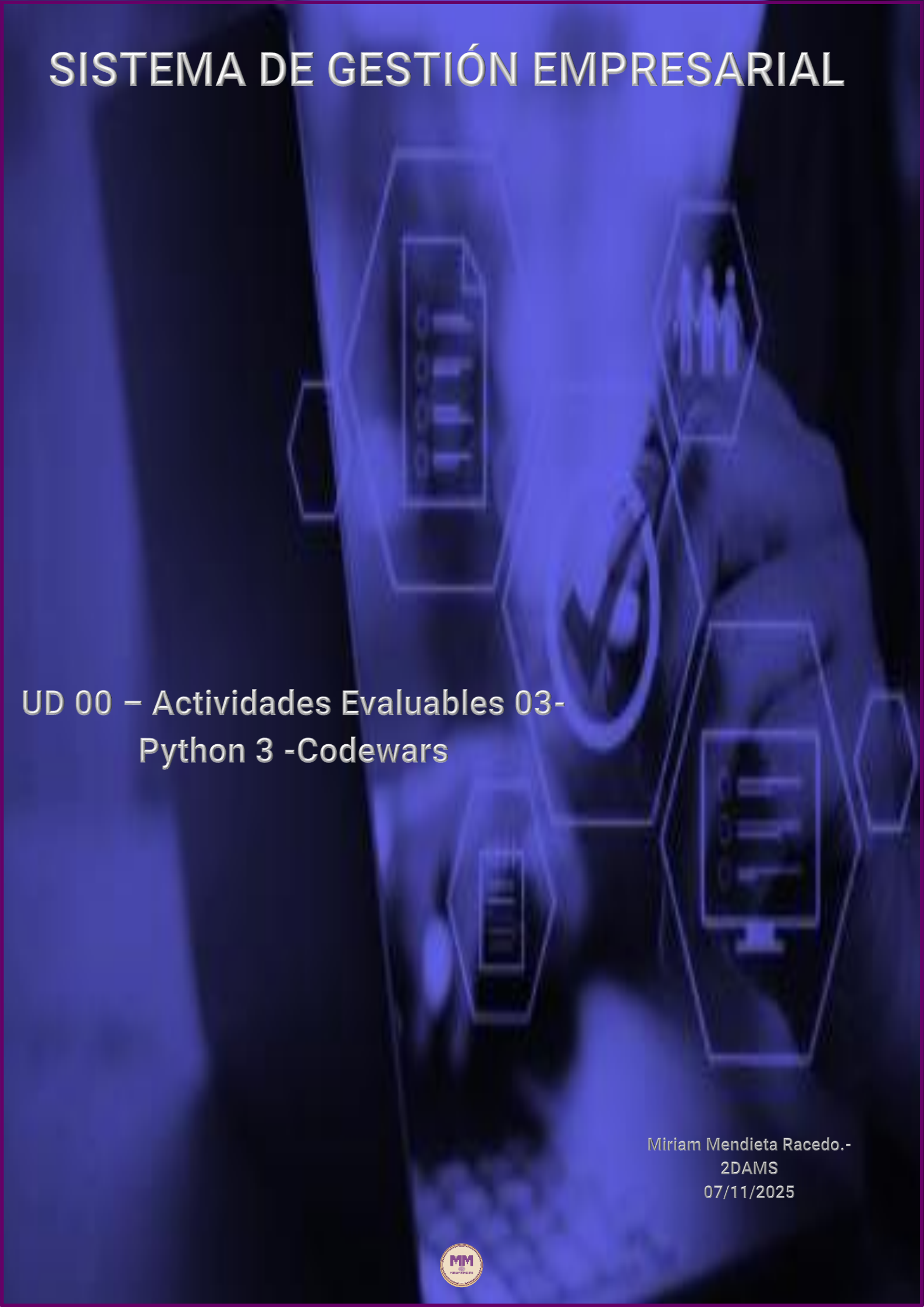
SISTEMA DE GESTIÓN EMPRESARIAL



UD 00 – Actividades Evaluables 03- Python 3 -Codewars

Miriam Mendieta Racedo.-
2DAMS
07/11/2025

ÍNDICE

INTRODUCCIÓN **1**

KATA_1 2
KATA_2 4
KATA_3 5
KATA_4 7
KATA_5 9
KATA_6 11
KATA_7 13
KATA_8 14
KATA_9 16
KATA_10 18
KATA_11 20
KATA_12 21
KATA_13 23
KATA_14 25
KATA_16 28
KATA_17 30
KATA_18 32
KATA_19 34
KATA_20 36

BIBLIOGRAFIA **38**

INTRODUCCIÓN

Este proyecto tiene como finalidad demostrar en forma individual, la resolución de problemas algorítmicos, dominio del lenguaje de programación Python y la capacidad para comprender, adaptar, documentar y mejorar código a partir de indicaciones específicas.

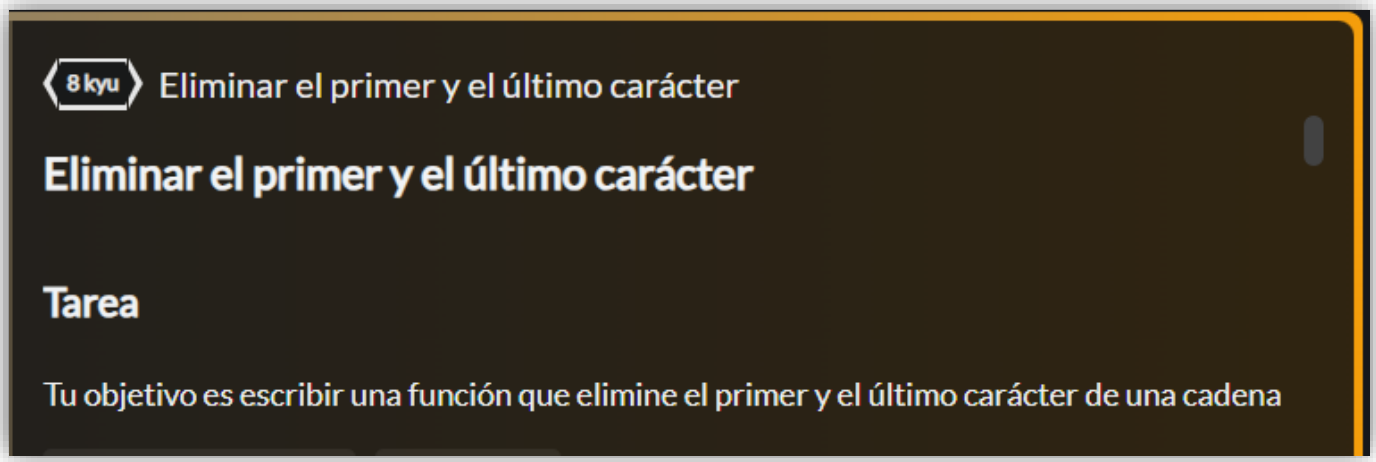
Para ello, se utilizará la plataforma CodeWars, un entorno de aprendizaje colaborativo donde programadores de todo el mundo practican sus habilidades resolviendo desafíos conocidos como katas. El término kata, tomado del mundo de las artes marciales, hace referencia a ejercicios repetitivos diseñados para perfeccionar técnicas. En el contexto de la programación, una kata es un problema concreto que debe resolverse mediante código, fomentando el pensamiento lógico, la creatividad y la mejora continua.

Se deberá completar un total de 15 katas utilizando Python: 10 de nivel 8 kyu (los más sencillos, ideales para afianzar conceptos básicos) y al menos 5 de nivel 7 kyu (de dificultad media, que requieren mayor análisis y precisión).

https://www.codewars.com/users/miriammendieta



KATA_1



Tarea Eliminar el primer y el último carácter

Tu objetivo es escribir una función que elimine el primer y el último carácter de una cadena de caracteres. Se te proporciona un parámetro: la cadena original.
Importante: Su función debe poder manejar cadenas de cualquier tipo length ≥ 2de caracteres. Para cadenas con exactamente 2caracteres, devuelva una cadena vacía.
Ejemplos
'eloquent' --> 'loquen'
'country' --> 'ountr'
'person' --> 'erso'
'ab' --> '' (empty string)
'xyz' --> 'y'
Requisitos

- La cadena de entrada siempre tendrá al menos 2 caracteres
- Para cadenas con exactamente 2 caracteres, devuelve una cadena vacía.
- Para cadenas con 3 o más caracteres, elimina el primer y el último carácter.
- La función debe manejar cadenas que contengan letras, números y caracteres especiales.

Casos de prueba
Su solución será probada contra:

- Funcionalidad básica con palabras comunes
- Casos límite con cadenas de 2 y 3 caracteres
- Cadenas que contienen números y caracteres especiales
- casos de prueba aleatorios de longitud variable

Solución

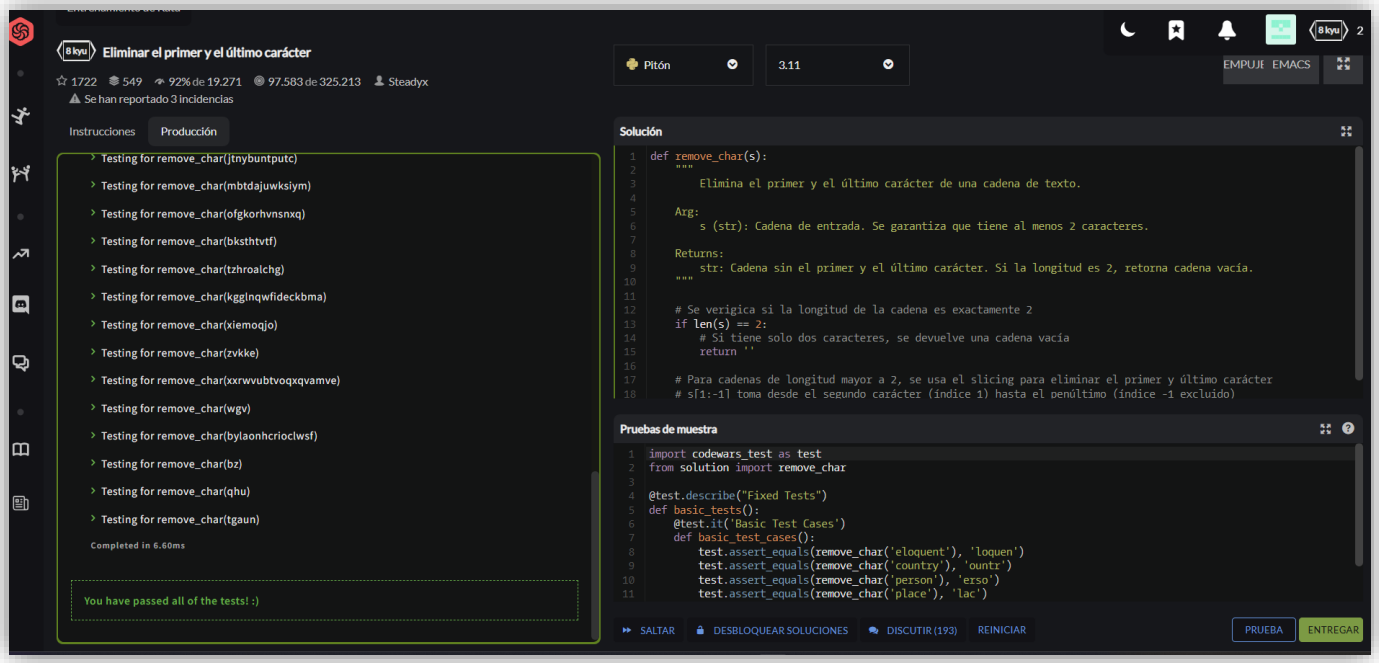
```python
def remove_char(s):
    """
    Elimina el primer y el último carácter de una cadena de texto.

    Arg:
        s (str): Cadena de entrada. Se garantiza que tiene al menos 2 caracteres.

    Returns:
        str: Cadena sin el primer y el último carácter. Si la longitud es 2, retorna cadena vacía.
    """

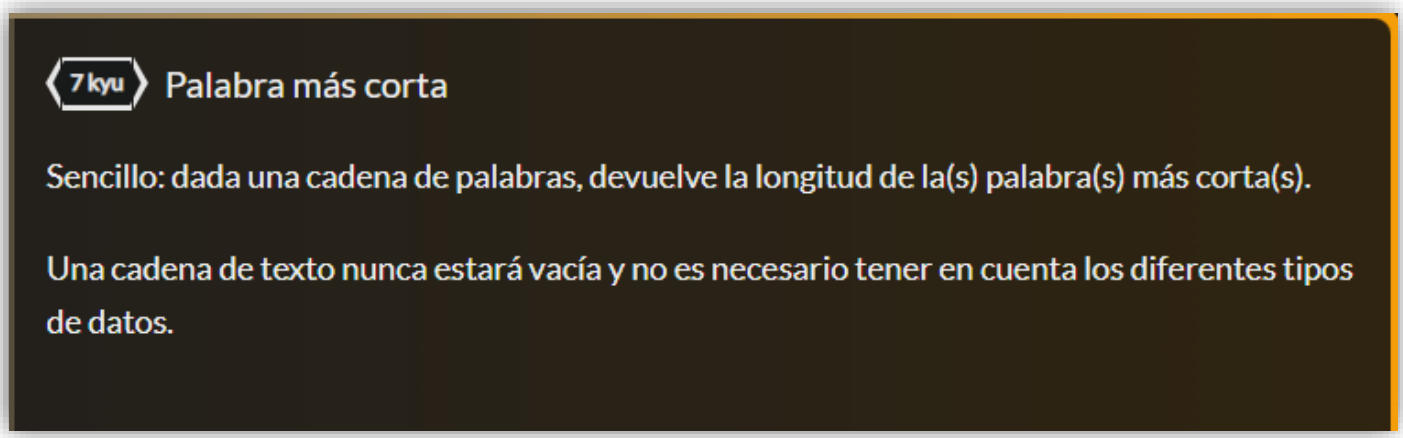
    # Se verifica si la longitud de la cadena es exactamente 2
    if len(s) == 2:
        # Si tiene solo dos caracteres, se devuelve una cadena vacía
        return ''

    # slicing para eliminar el primer y último carácter
    # s[1:-1] toma desde el segundo carácter (índice 1) hasta el penúltimo (índice -1 excluido)
    return s[1:-1]
```



KATA_2

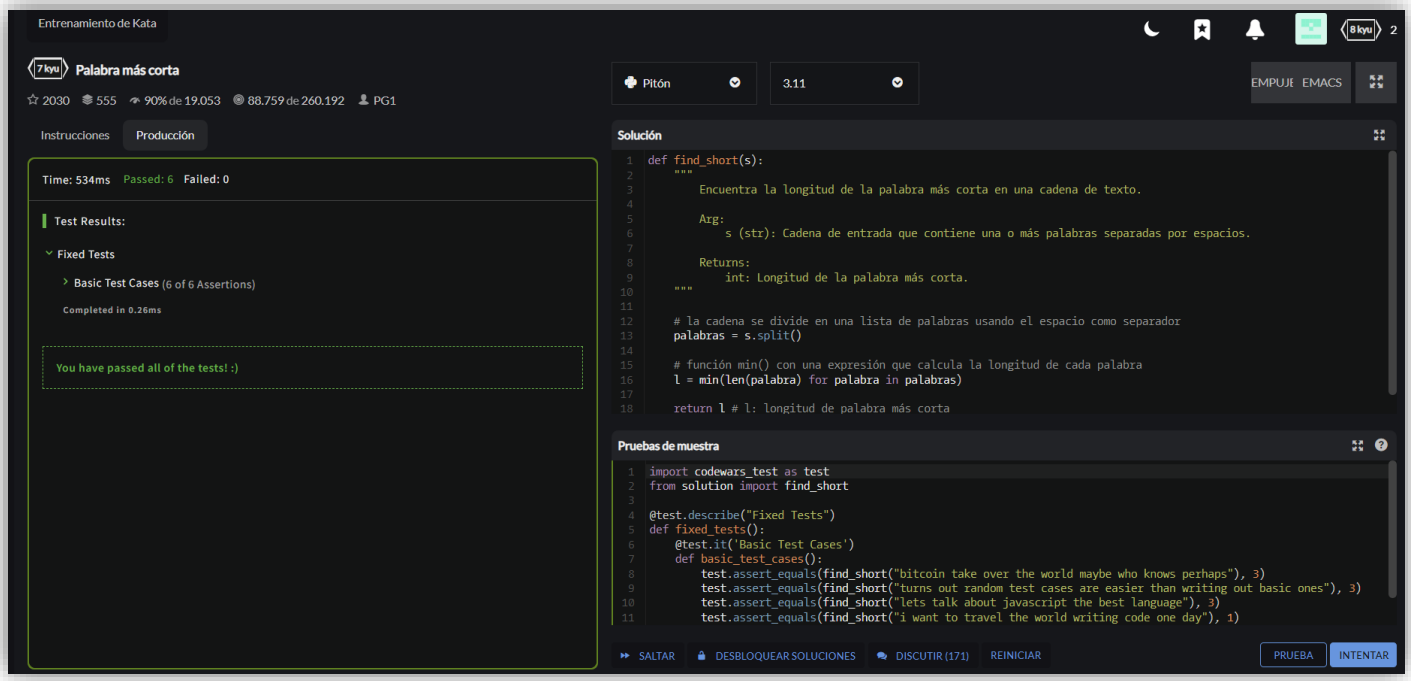
Sencillo: dada una cadena de palabras, devuelve la longitud de la(s) palabra(s) más corta(s).
Una cadena de texto nunca estará vacía y no es necesario tener en cuenta los diferentes tipos de datos.



Solución

```python
def find_short(s):
    """
        Encuentra la longitud de la palabra más corta en una cadena de texto.
        Arg:
            s (str): Cadena de entrada que contiene una o más palabras separadas por espacios.
        Returns:
            int: Longitud de la palabra más corta.
    """
    # la cadena se divide en una lista de palabras usando el espacio como separador
    palabras = s.split()
    # función min() con una expresión que calcula la longitud de cada palabra
    l = min(len(palabra) for palabra in palabras)

    return l # l: longitud de palabra más corta
```



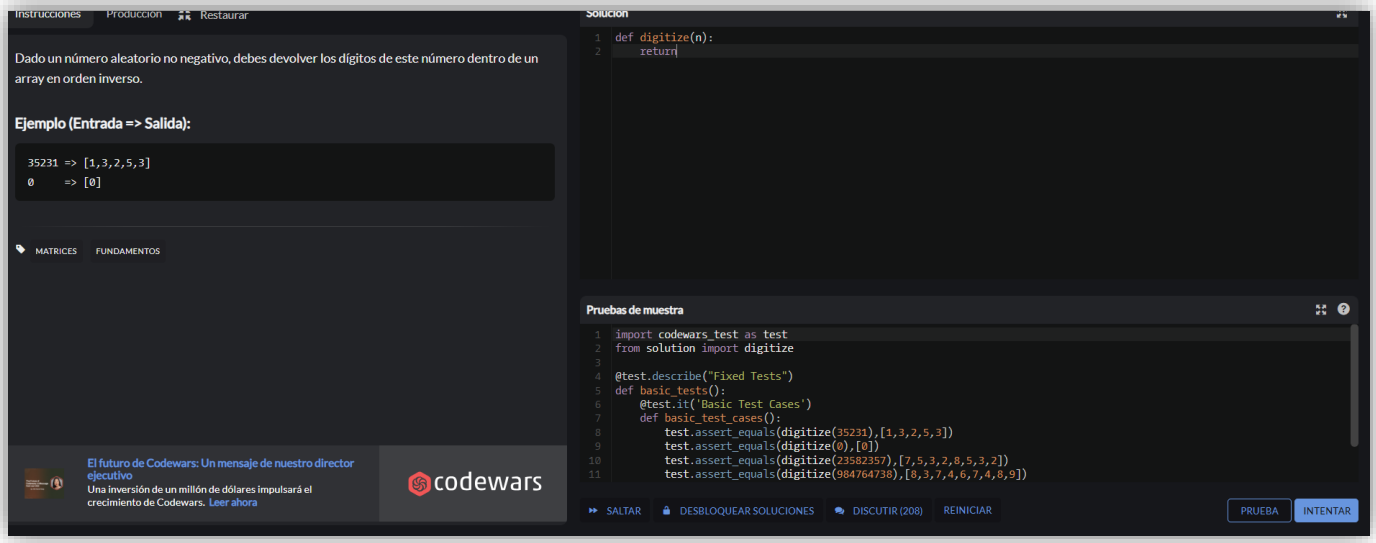
KATA_3

Dado un número aleatorio no negativo, debes devolver los dígitos de este número dentro de un array en orden inverso.

Ejemplo (Entrada => Salida):

35231 => [1,3,2,5,3]

0 => [0]



Solución

```python
def digitize(n):
    """
    Convierte un número entero no negativo en una lista de sus dígitos en orden inverso.

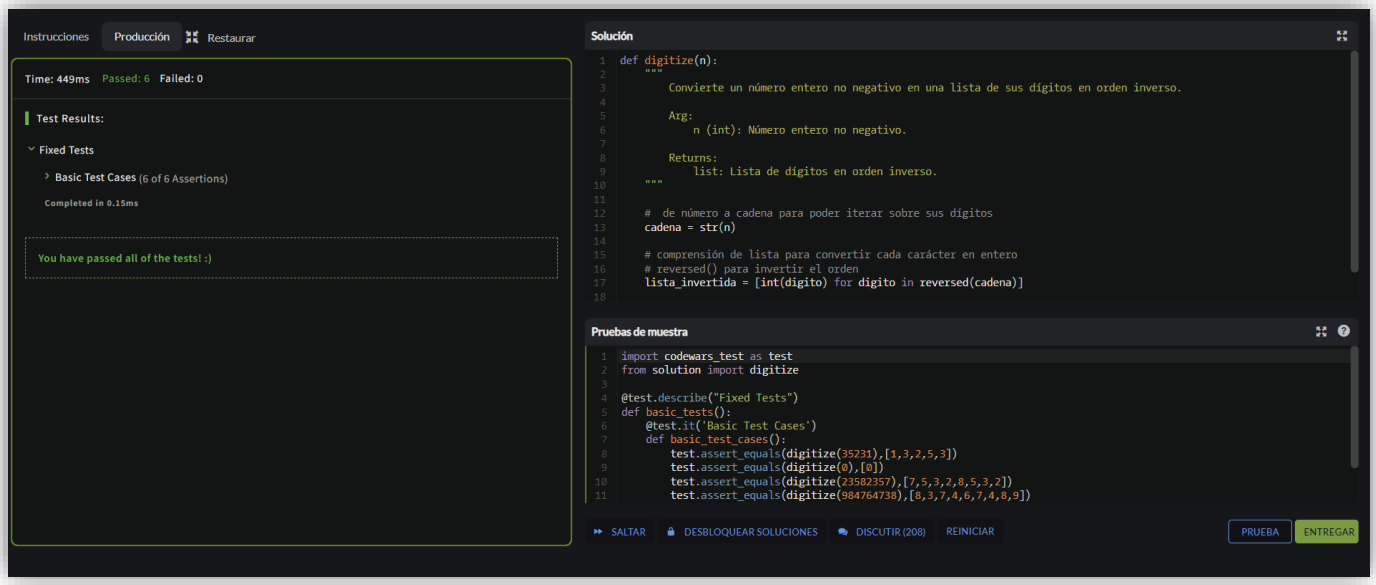
    Arg:
        n (int): Número entero no negativo.

    Returns:
        list: Lista de dígitos en orden inverso.
    """

    #  de número a cadena para poder iterar sobre sus dígitos
    cadena = str(n)

    # comprensión de lista para convertir cada carácter en entero
    # reversed() para invertir el orden
    lista_invertida = [int(digito) for digito in reversed(cadena)]

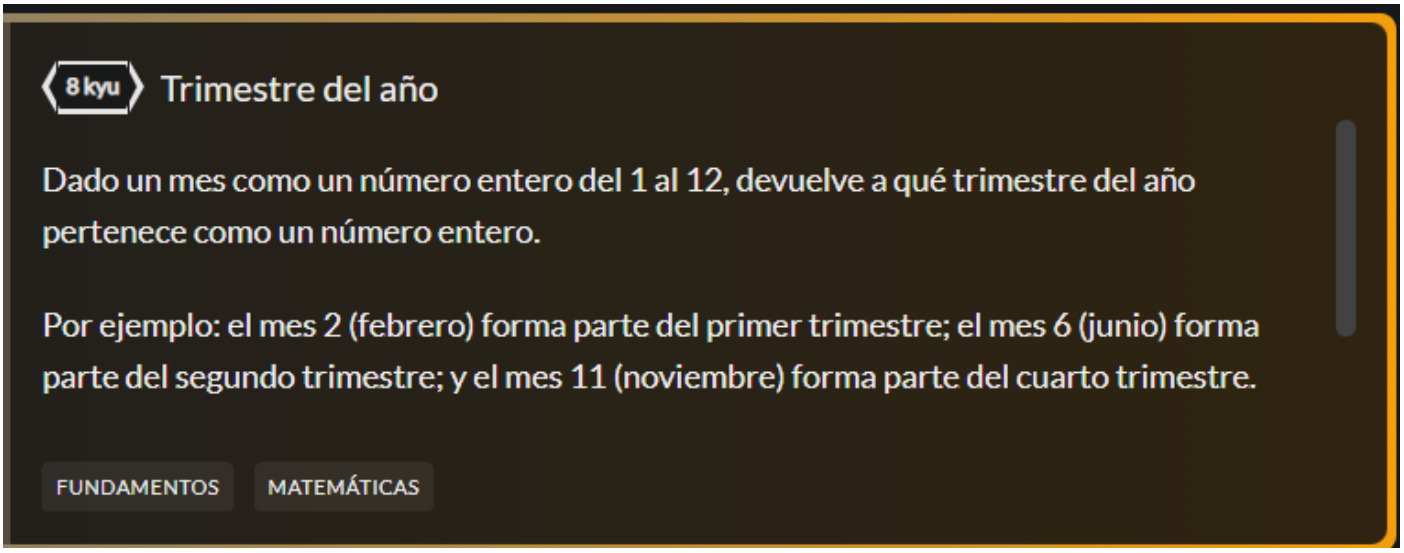
    # Devuelve la lista resultante
    return lista_invertida
```



KATA_4

Dado un mes como un número entero del 1 al 12, devuelve a qué trimestre del año pertenece como un número entero.
Por ejemplo: el mes 2 (febrero) forma parte del primer trimestre; el mes 6 (junio) forma parte del segundo trimestre; y el mes 11 (noviembre) forma parte del cuarto trimestre.

Restricción:
1 <= month <= 12



Solución

```python
def get_quarter(month: int) -> int:
    """
    Devuelve el trimestre del año al que pertenece un mes dado.

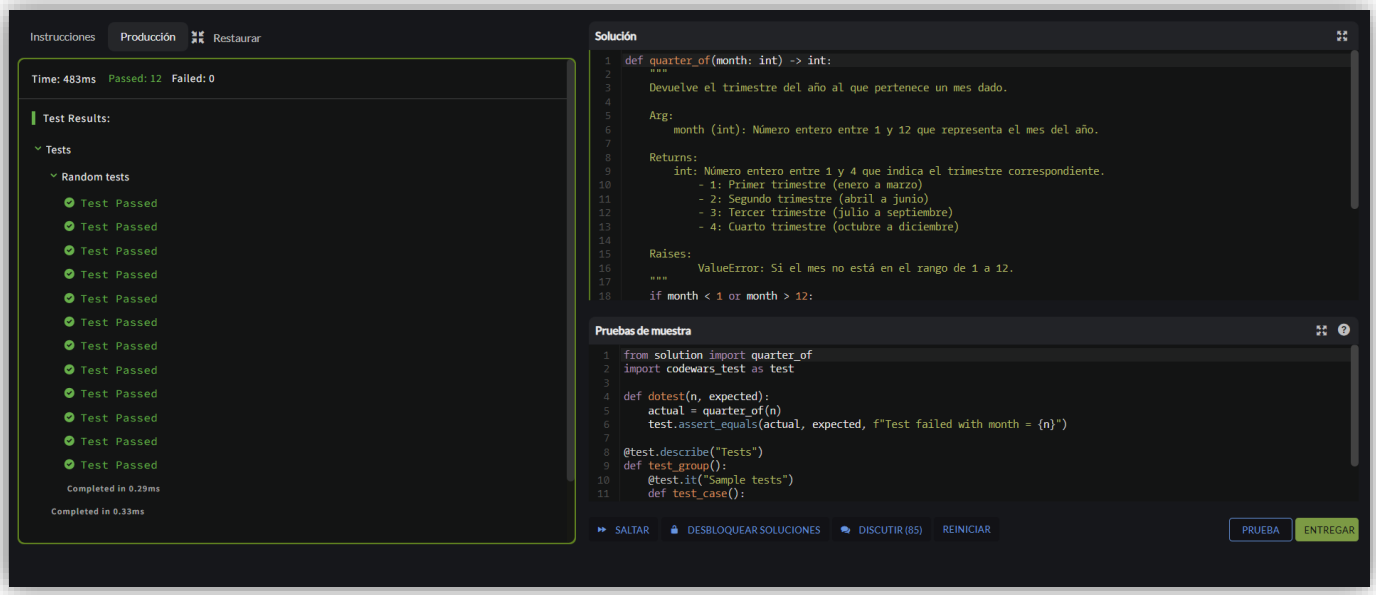
    Arg:
        month (int): Número entero entre 1 y 12 que representa el mes del año.

    Returns:
        int: Número entero entre 1 y 4 que indica el trimestre correspondiente.
            - 1: Primer trimestre (enero a marzo)
            - 2: Segundo trimestre (abril a junio)
            - 3: Tercer trimestre (julio a septiembre)
            - 4: Cuarto trimestre (octubre a diciembre)
```

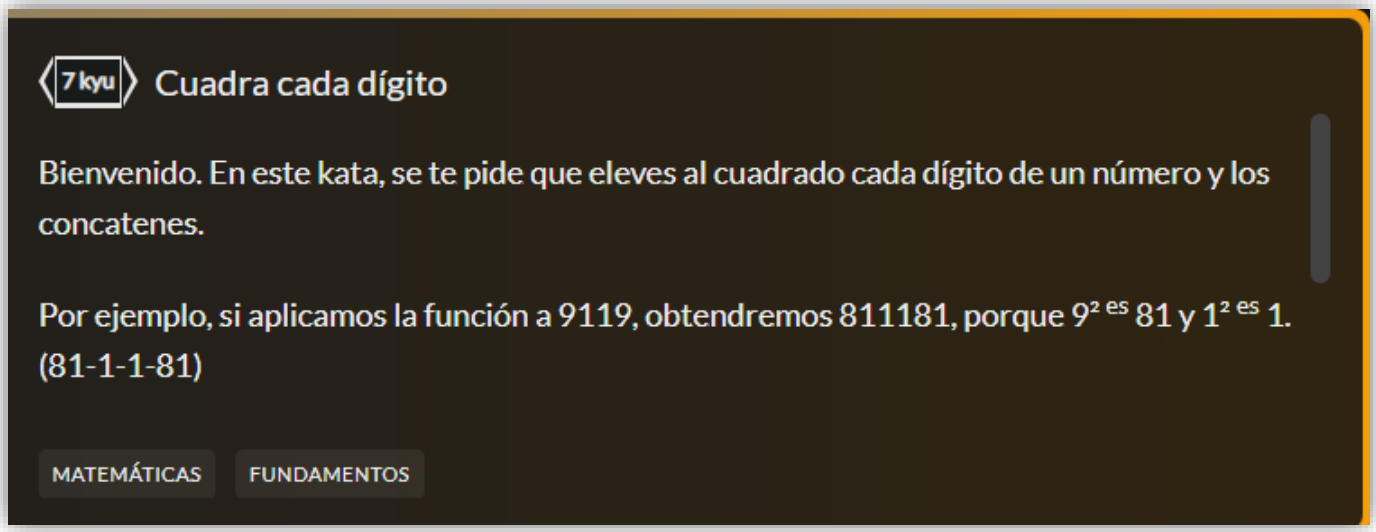
```
    Raises:
        ValueError: Si el mes no está en el rango de 1 a 12.
    """
    if month < 1 or month > 12:
        raise ValueError("El mes debe estar entre 1 y 12")

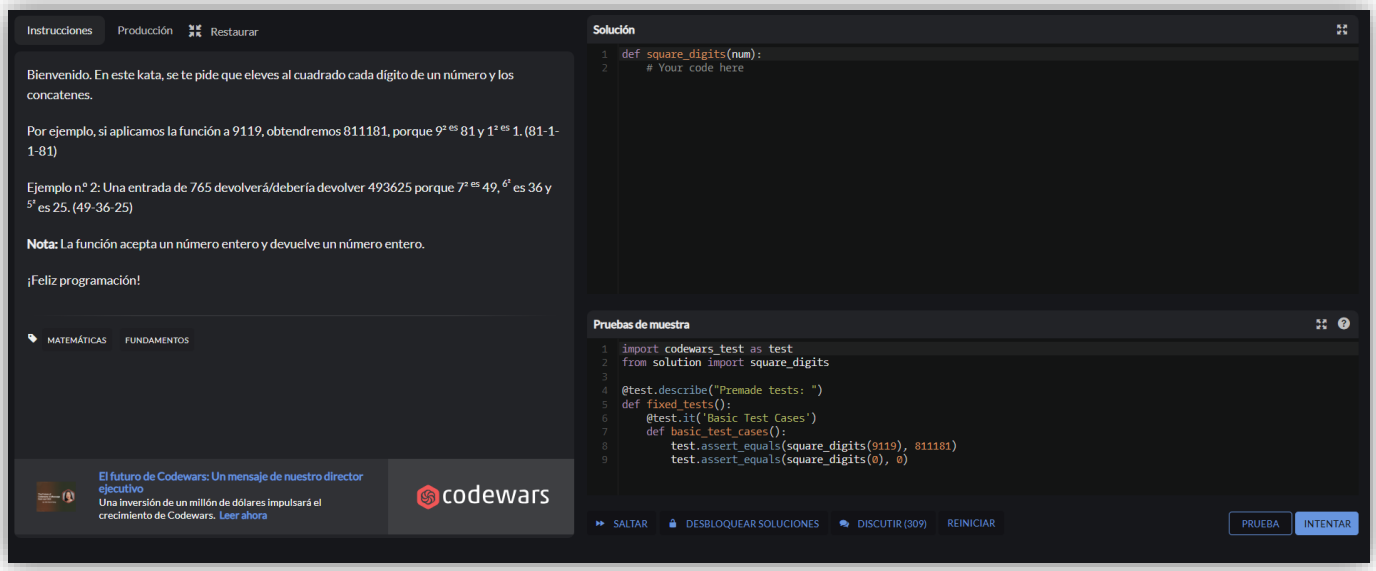
    #primer trimestre: meses 1, 2, 3
    if month <= 3:
        return 1
    #segundo trimestre: meses 4, 5, 6
    elif month <= 6:
        return 2
    #tercer trimestre: meses 7, 8, 9
    elif month <= 9:
        return 3

    #cuarto trimestre: meses 10, 11, 12
    else:
        return 4
```



KATA_5





Solución

```python
def square_digits(num: int) -> int:
    """
    Eleva al cuadrado cada dígito de un número entero y concatena los resultados.

    Arg:
        num (int): Número entero positivo.

    Return:
        int: Número formado por la concatenación de los cuadrados de cada dígito.
```

```
        Ejemplos:
            - square_digits(9119) → 811181
            - square_digits(765) → 493625
    """

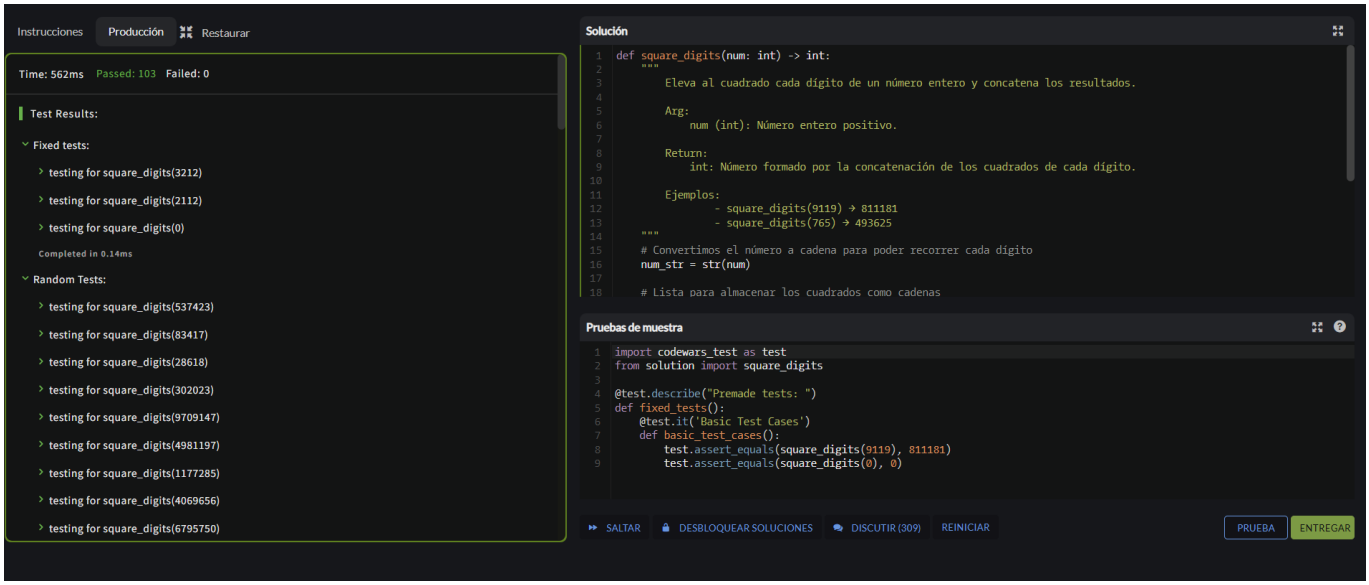
    # Convertimos el número a cadena para poder recorrer cada dígito
    num_str = str(num)

    # Lista para almacenar los cuadrados como cadenas
    squared_str_list = []

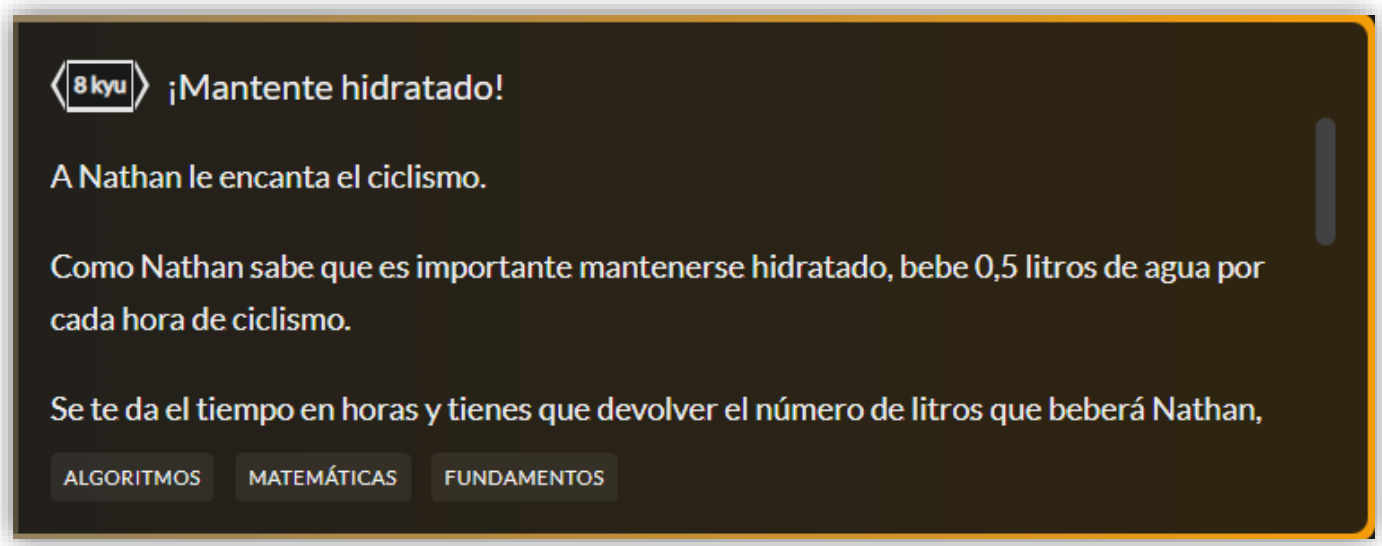
    # Recorremos cada carácter (dígito) en la cadena
    for digit in num_str:
        # Convertimos el carácter a entero, lo elevamos al cuadrado y lo convertimos de nuevo a cadena
        squared_digit = str(int(digit) ** 2)
        # Lo añadimos a la lista
        squared_str_list.append(squared_digit)

    # Unimos todos los cuadrados en una sola cadena
    result_str = ''.join(squared_str_list)

    # Convertimos el resultado final a entero y lo devolvemos
    return int(result_str)
```



KATA_6



A Nathan le encanta el ciclismo.

Como Nathan sabe que es importante mantenerse hidratado, bebe 0,5 litros de agua por cada hora de ciclismo.Se te da el tiempo en horas y tienes que devolver el número de litros que beberá Nathan, redondeado hacia abajo .

Por ejemplo:

time = 3 ----> litres = 1

time = 6.7---> litres = 3

time = 11.8--> litres = 5

El problema se puede modelar como una **función lineal** de la forma:

$$L(t)=[0.5 \cdot t]$$

donde:

- t es el tiempo en horas (número real positivo)
- L(t) es la cantidad de litros de agua que Nathan debe beber
- La cantidad de agua que bebe Nathan es **directamente proporcional** al tiempo que pasa montando en bicicleta. Esto se expresa como:
- litros $= 0.5 \cdot$ horas
- Este tipo de relación es una función lineal con pendiente $m = 0.5$, lo que significa que por cada hora adicional, Nathan bebe medio litro más.

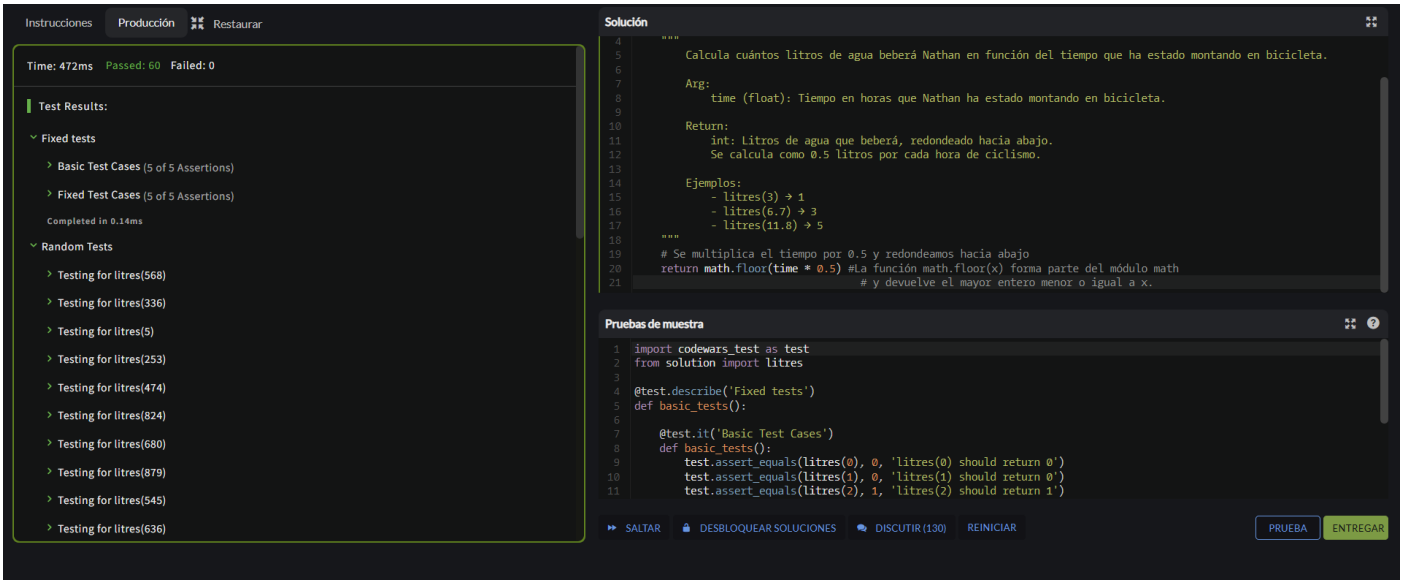
Solución

```python
import math
def litres(time: float) -> int:
    """
    Calcula cuántos litros de agua beberá Nathan en función del tiempo que ha estado montando en bicicleta.

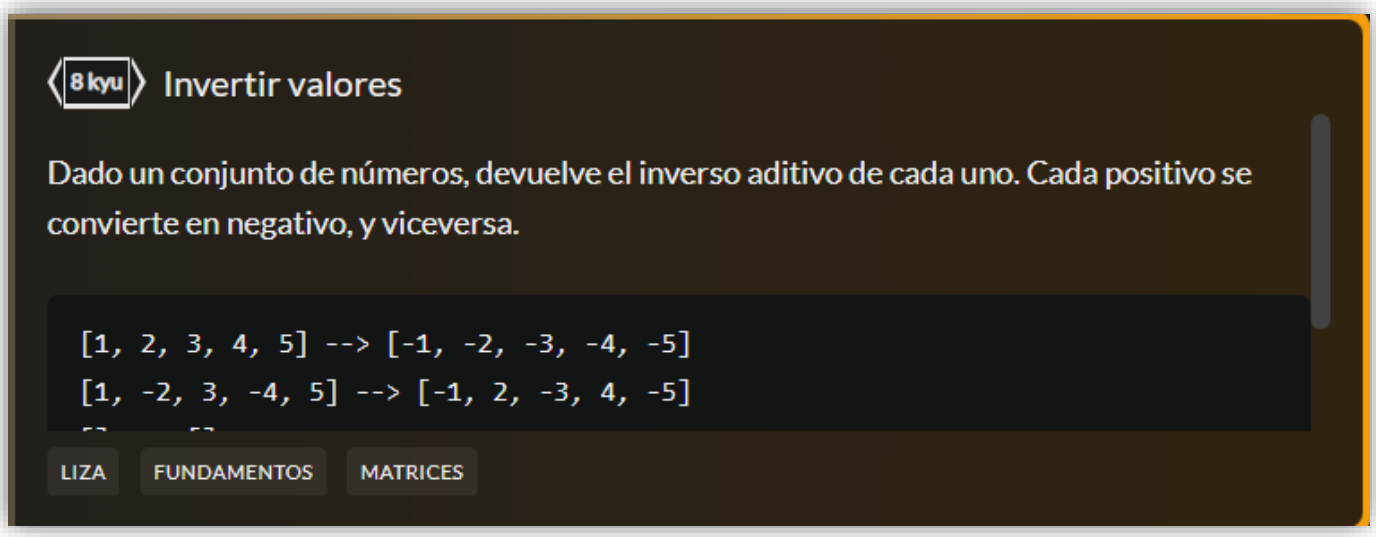
    Arg:
        time (float): Tiempo en horas que Nathan ha estado montando en bicicleta.

    Return:
        int: Litros de agua que beberá, redondeado hacia abajo.
        Se calcula como 0.5 litros por cada hora de ciclismo.

    Ejemplos:
        - litres(3) → 1
        - litres(6.7) → 3
        - litres(11.8) → 5
    """
    # Se multiplica el tiempo por 0.5 y redondeamos hacia abajo
    return math.floor(time * 0.5) #La función math.floor(x) forma parte del módulo math
                                  # y devuelve el mayor entero menor o igual a x.
```



KATA_7



Dado un conjunto de números, devuelve el inverso aditivo de cada uno. Cada positivo se convierte en negativo, y viceversa.

[1, 2, 3, 4, 5] --> [-1, -2, -3, -4, -5]

[1, -2, 3, -4, 5] --> [-1, 2, -3, 4, -5]

[ ] --> [ ]

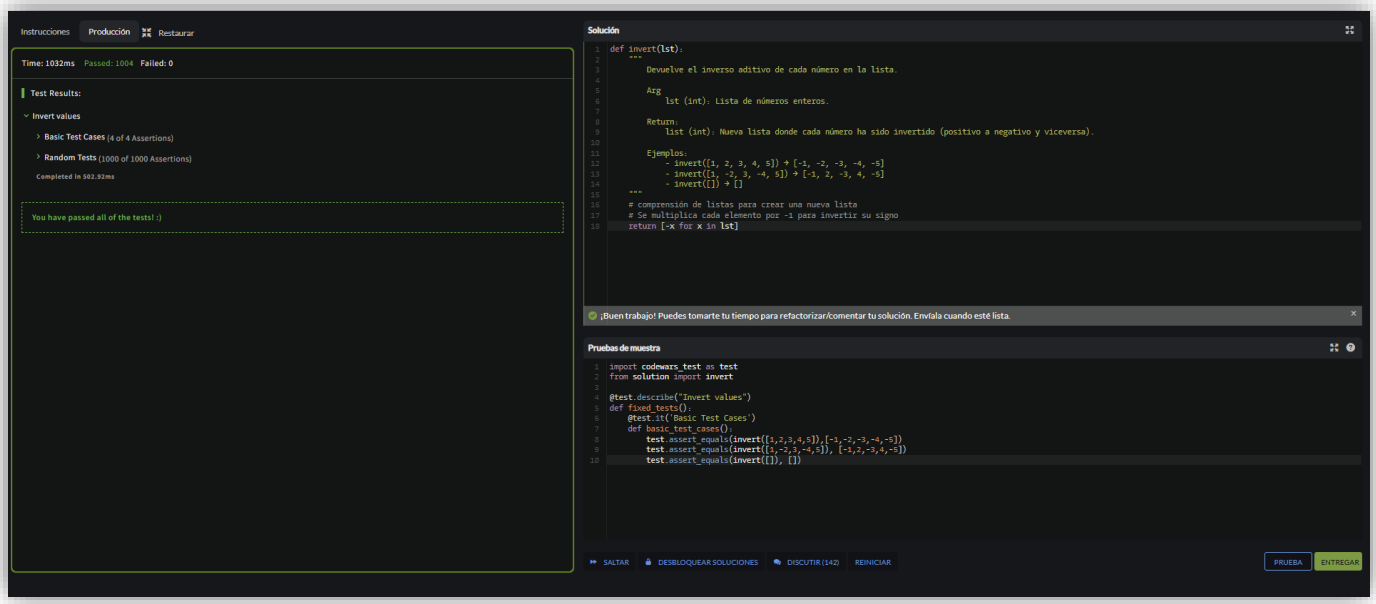
Puede asumir que todos los valores son enteros. No modifique el array de entrada.

Solución

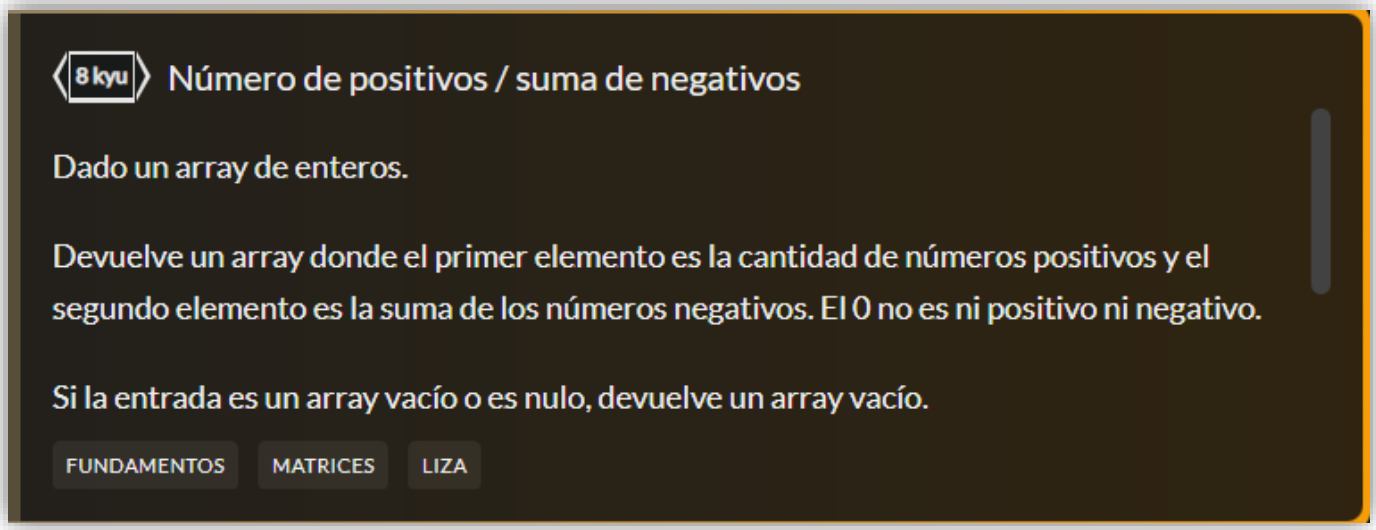
```
def invert(lst):
    """
        Devuelve el inverso aditivo de cada número en la lista.
        Arg
            lst (int): Lista de números enteros.
         Return:
            list (int): Nueva lista donde cada número ha sido invertido (positivo a negativo y viceversa).
        Ejemplos:
            - invert([1, 2, 3, 4, 5]) → [-1, -2, -3, -4, -5]
            - invert([1, -2, 3, -4, 5]) → [-1, 2, -3, 4, -5]
```

```
        - invert([]) → []
    """

    # comprensión de listas para crear una nueva lista
    # Se multiplica cada elemento por -1 para invertir su signo
    return [-x for x in lst]
```

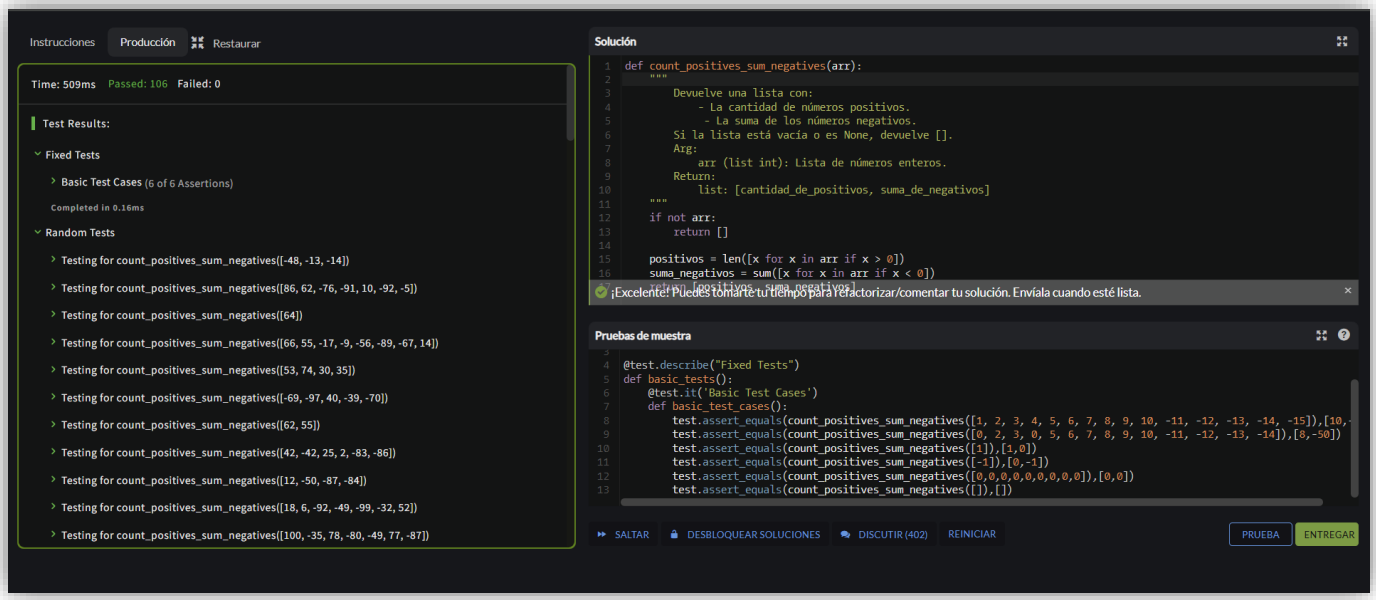


KATA_8

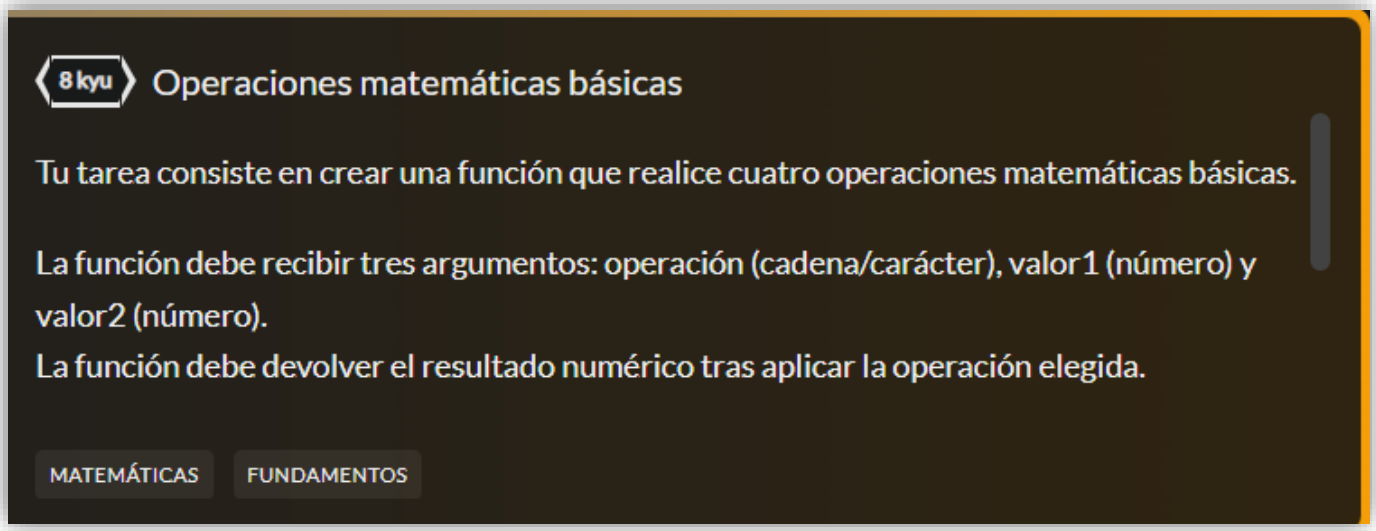


Solución

```python
def count_positives_sum_negatives(arr):
    """
        Devuelve una lista con:
            - La cantidad de números positivos.
            - La suma de los números negativos.
        Si la lista está vacía o es None, devuelve [].
        Arg:
            arr (list int): Lista de números enteros.
        Return:
            list: [cantidad_de_positivos, suma_de_negativos]
    """
# Verifica si la lista está vacía o es None (nula).Si es así, devuelve una lista vacía como resultado.
if not arr:
    return [ ]
# Se crea una nueva lista con los elementos positivos de arr usando comprensión de listas. Luego calcula cuántos hay con len().
positivos = len([x for x in arr if x > 0])
# nueva lista con los elementos negativos de arr usando comprensión de listas. Luego suma todos esos elementos con sum().
suma_negativos = sum([x for x in arr if x < 0])
# Devuelve una lista con dos elementos: cantidad de positivos y la suma de los negativos.
return [positivos, suma_negativos]
```



KATA_9



Tu tarea consiste en crear una función que realice cuatro operaciones matemáticas básicas.
La función debe recibir tres argumentos: operación (cadena/carácter), valor1 (número) y valor2 (número).
La función debe devolver el resultado numérico tras aplicar la operación elegida.
Ejemplos(Operador, valor1, valor2) --> salida
('+', 4, 7) --> 11

('-', 15, 18) --> -3

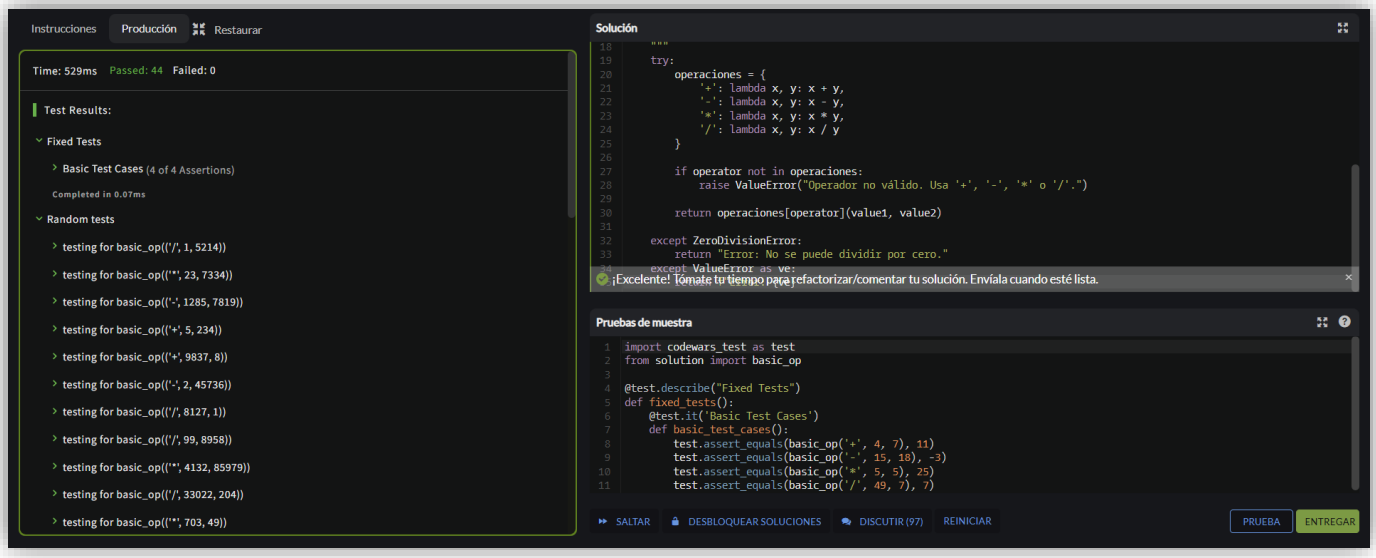
('*', 5, 5) --> 25

('/', 49, 7) --> 7

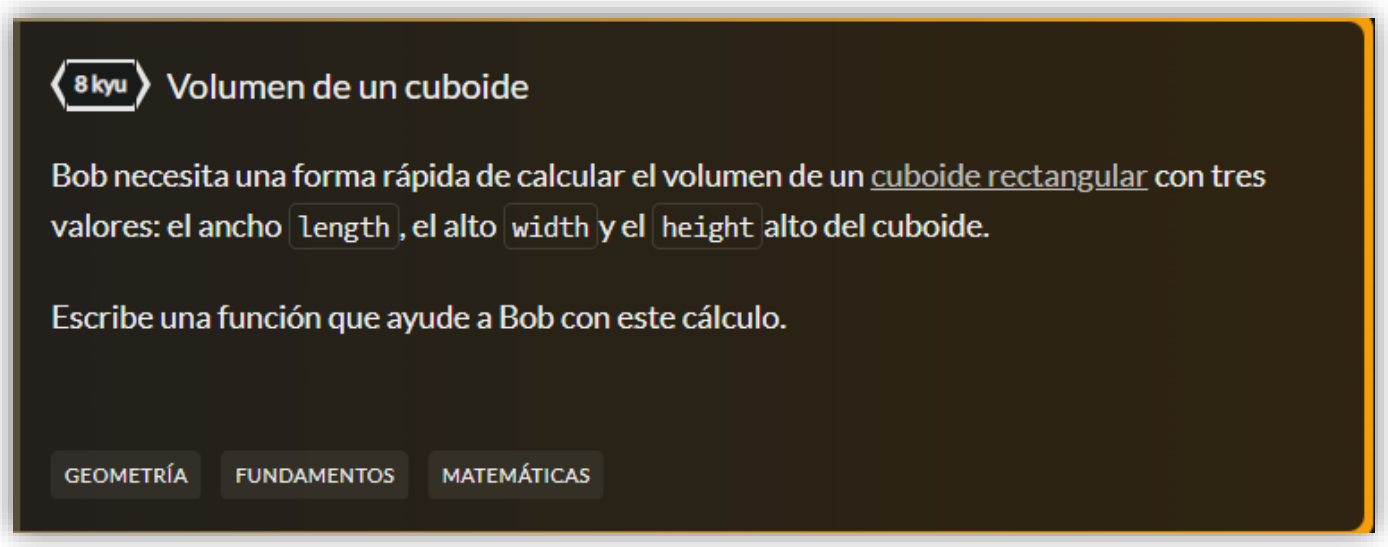
Solución

```
def basic_op(operator, value1, value2):
    """
    Realiza una operación matemática básica entre dos valores.
    Captura errores como división por cero o operador inválido.
    Arg:
        operator (str): Operador matemático ('+', '-', '*', '/').
        value1 (float o int): Primer operando.
        value2 (float o int): Segundo operando.
```

```
Return:
    float o int: Resultado de aplicar la operación entre value1 y value2.
Raises:
    ValueError: Si el operador no es válido.
    ZeroDivisionError: Si se intenta dividir por cero.
Ejemplos:
    - basic_op('+', 4, 7) → 11
   - basic_op('-', 15, 18) → -3
   - basic_op('*', 5, 5) → 25
   - basic_op('/', 49, 7) → 7.0
"""
try:
    operaciones = {
        '+': lambda x, y: x + y,
        '-': lambda x, y: x - y,
        '*': lambda x, y: x * y,
        '/': lambda x, y: x / y
    }
    if operator not in operaciones:
        raise ValueError("Operador no válido. Usa '+', '-', '*' o '/'.")
    return operaciones[operator](value1, value2)
except ZeroDivisionError:
    return "Error: No se puede dividir por cero."
except ValueError as ve:
    return f"Error: {ve}"
```



KATA_10



Un **cuboide rectangular** (también llamado paralelepípedo) es una figura tridimensional con seis caras rectangulares. Su volumen se calcula como:

$$V = L \cdot W \cdot H$$

donde:

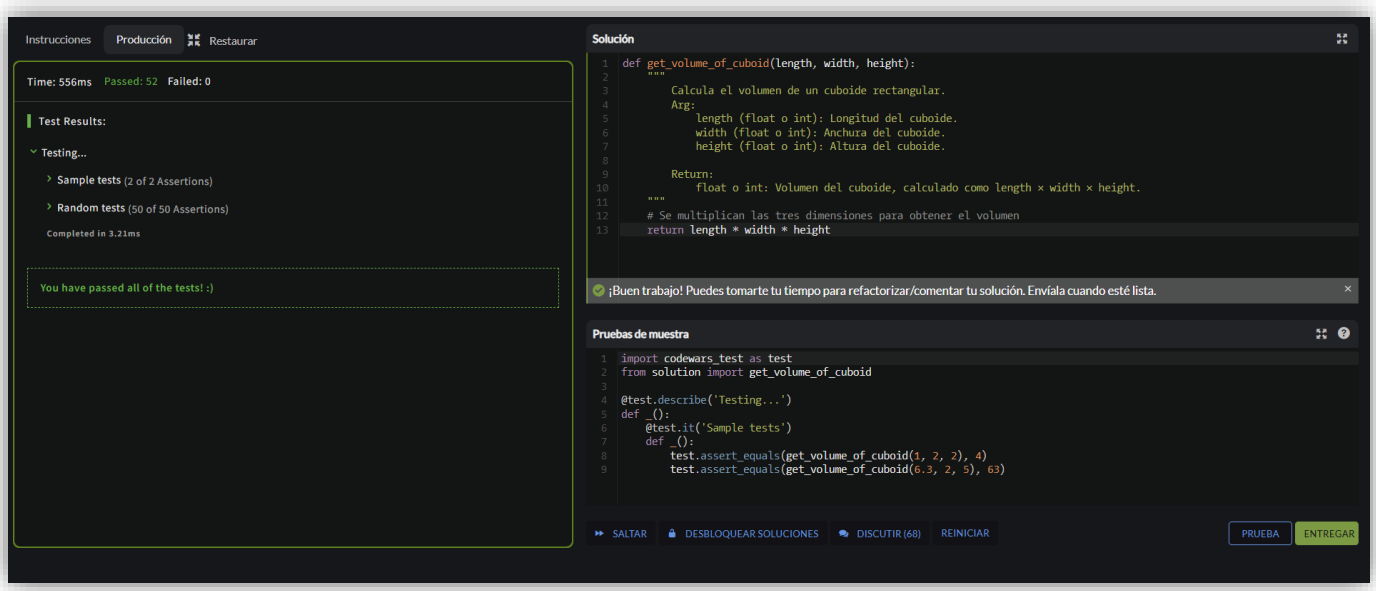
- L es la longitud (length)
- W es la anchura (width)
- H es la altura (height)
- V es el volumen, que representa el espacio tridimensional que ocupa el cuboide.

Este cálculo proviene de la fórmula general del volumen de un prisma rectangular, y se expresa en unidades cúbicas (por ejemplo, cm^3, m^3).

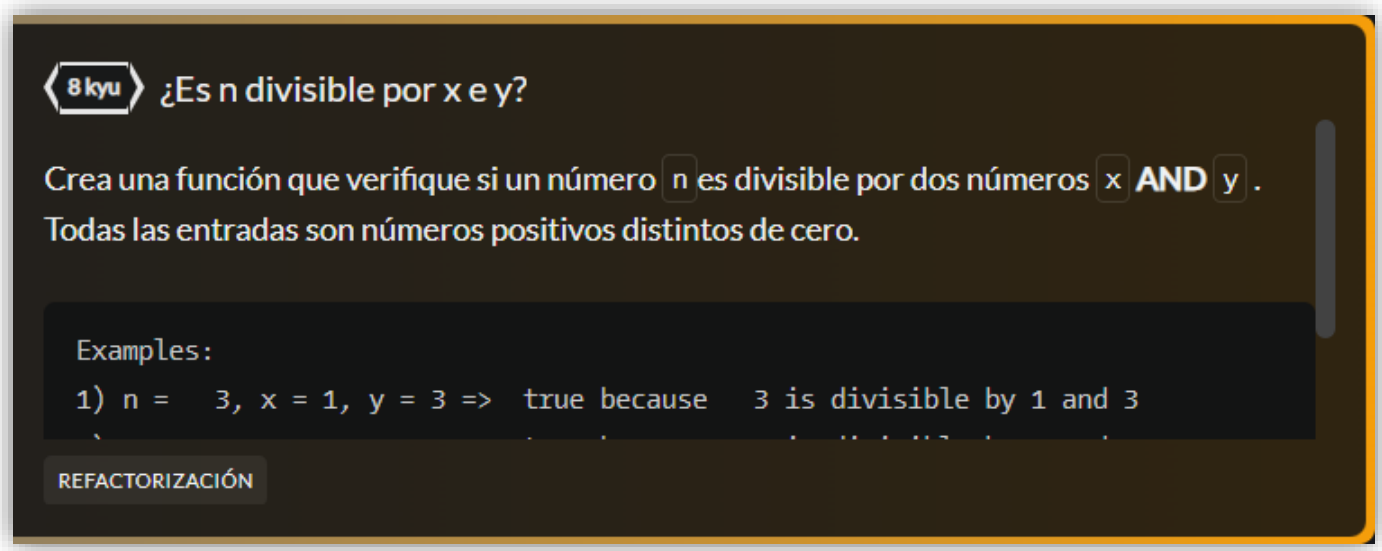
Solución

```
def get_volume_of_cuboid(length, width, height):
    """
        Calcula el volumen de un cuboide rectangular.
        Arg:
            length (float o int): Longitud del cuboide.
            width (float o int): Anchura del cuboide.
            height (float o int): Altura del cuboide.
```

```
    Return:
        float o int: Volumen del cuboide, calculado como length × width × height.
    """
    # Se multiplican las tres valores para obtener el volumen
    return length * width * height
```



KATA_11



- Un número $n$es **divisible** por otro número $d$si el **resto de la división** $n \bmod d = 0$.
- En este caso, verificamos dos condiciones simultáneamente:

$$n \bmod x = 0 y n \bmod y = 0$$

- Si ambas se cumplen, entonces *n es* divisible por X y por y

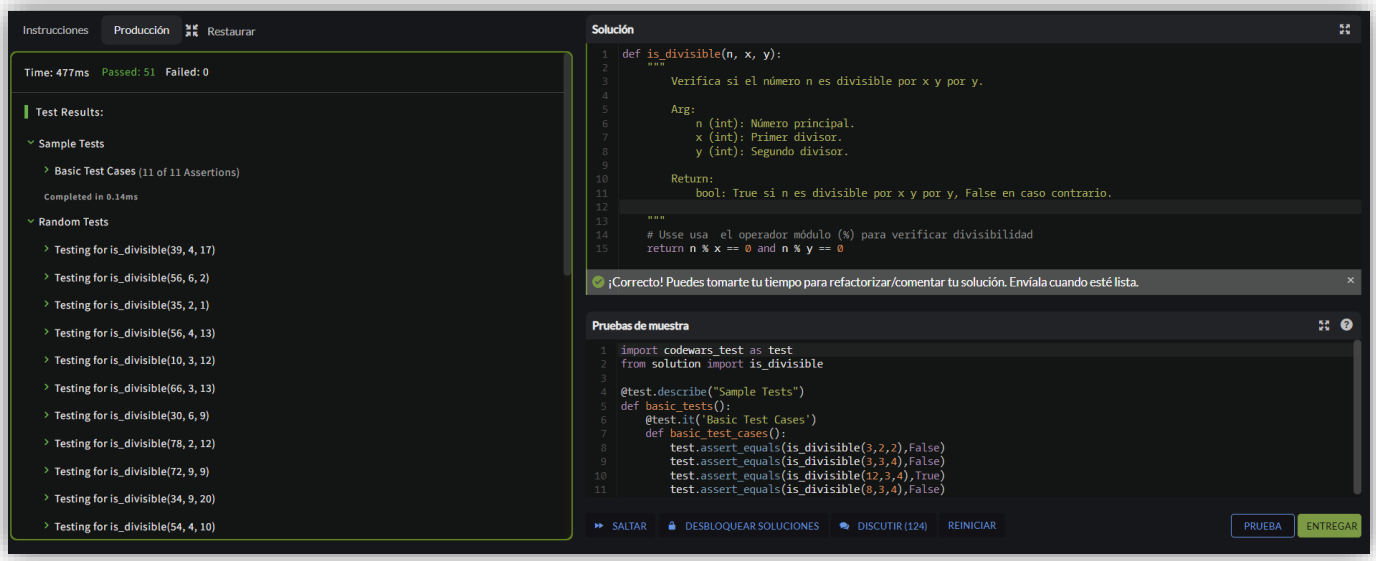
Solución

```python
def is_divisible(n, x, y):
    """
    Verifica si el número n es divisible por x y por y.
    Arg:
        n (int): Número principal.
        x (int): Primer divisor.
        y (int): Segundo divisor.
    Return:
        bool: True si n es divisible por x y por y, False en caso contrario.

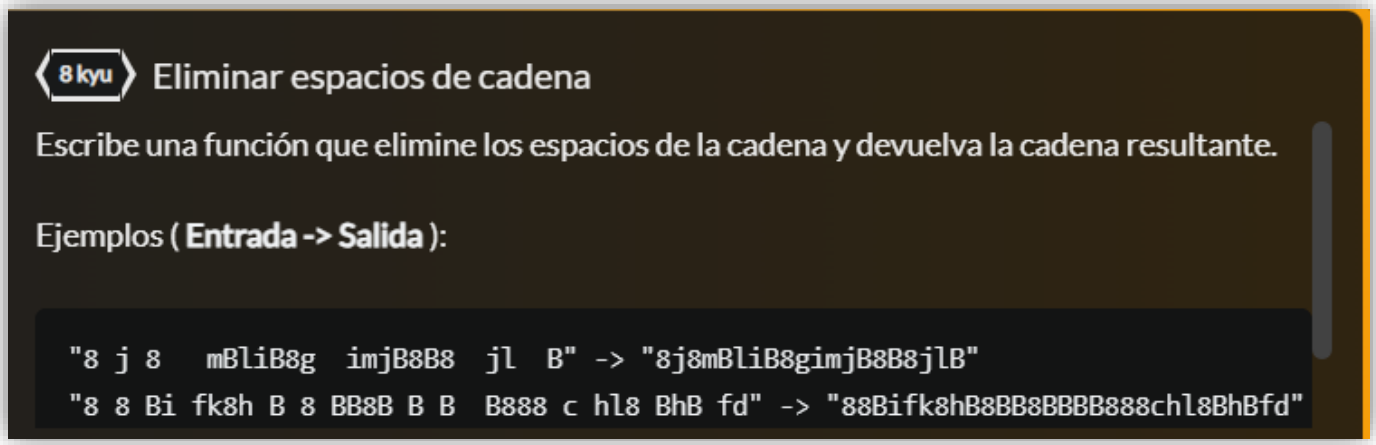
    Ejemplos:
```

```
    - is_divisible(3, 1, 3) → True
    - is_divisible(12, 2, 6) → True
    - is_divisible(100, 5, 3) → False
    - is_divisible(12, 7, 5) → False
"""

# Usse usa  el operador módulo (%) para verificar divisibilidad
return n % x == 0 and n % y == 0
```



KATA_12



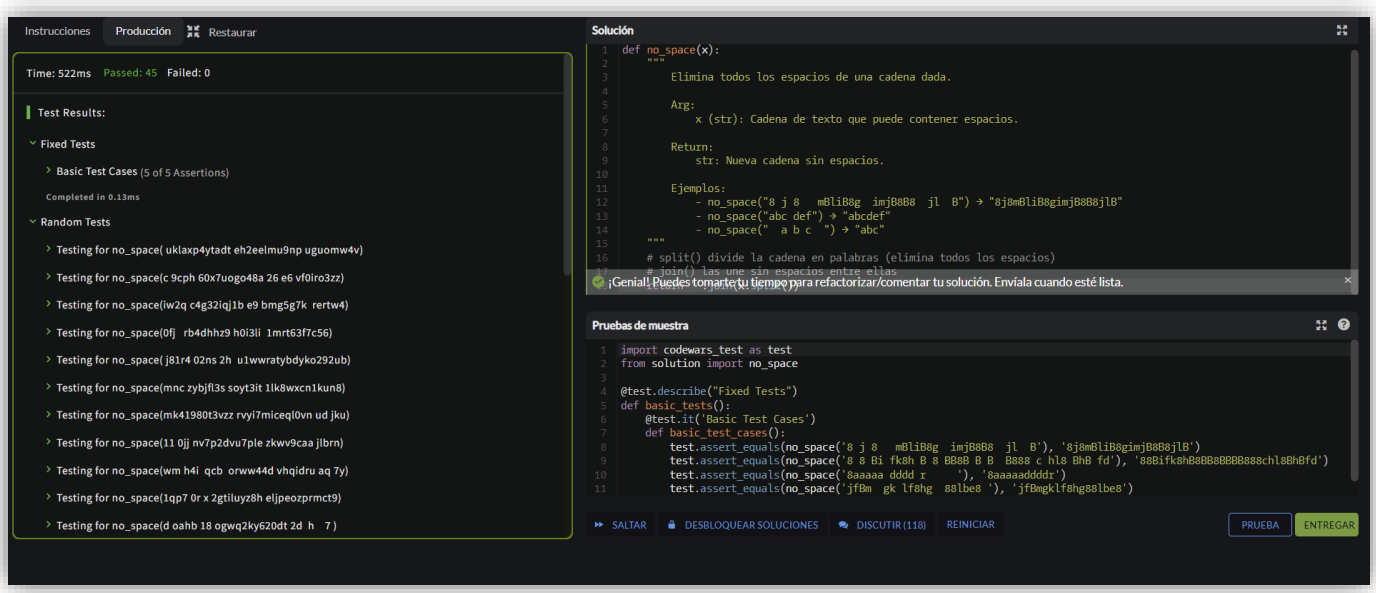
Solución

```python
def no_space(x):
    """
        Elimina todos los espacios de una cadena dada.

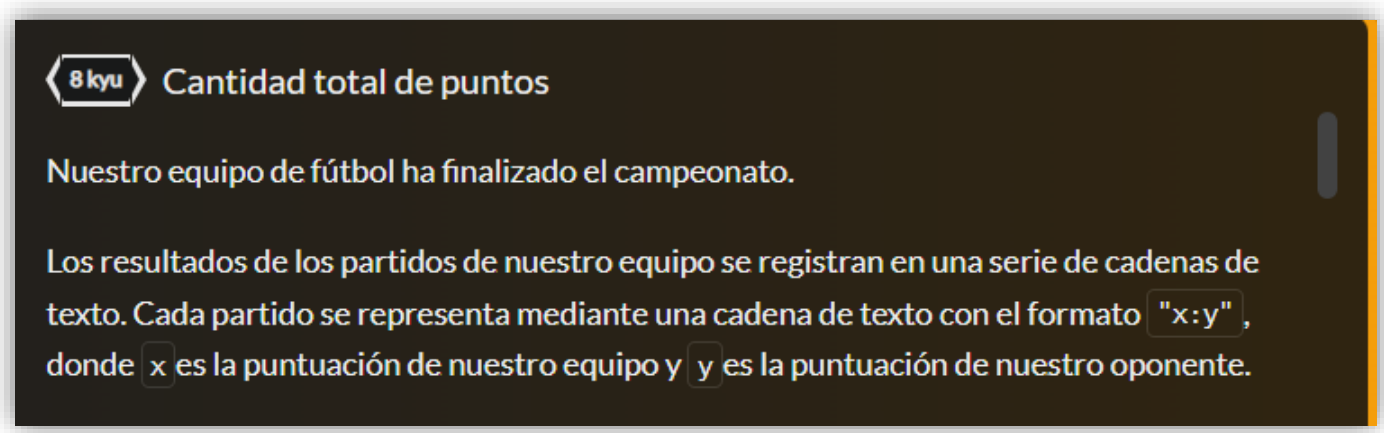
        Arg:
            x (str): Cadena de texto que puede contener espacios.

        Return:
            str: Nueva cadena sin espacios.

        Ejemplos:
            - no_space("8 j 8   mBliB8g  imjB8B8  jl  B") → "8j8mBliB8gimjB8B8jlB"
            - no_space("abc def") → "abcdef"
            - no_space("  a b c  ") → "abc"
    """
    # split( ) divide la cadena en palabras (elimina todos los espacios)
    # join( ) las une sin espacios entre ellas
    return ''.join(x.split())
```



KATA_13



Nuestro equipo de fútbol ha finalizado el campeonato.

Los resultados de los partidos de nuestro equipo se registran en una serie de cadenas de texto. Cada partido se representa mediante una cadena de texto con el formato "x:y", donde xes la puntuación de nuestro equipo y yes la puntuación de nuestro oponente.

Por ejemplo: ["3:1", "2:2", "0:1", ...]

Los puntos se otorgan por cada partido de la siguiente manera:

- Si x > y: 3 puntos (gana)
- Si x < y: 0 puntos (pérdida)
- Si x = y: 1 punto (empate)

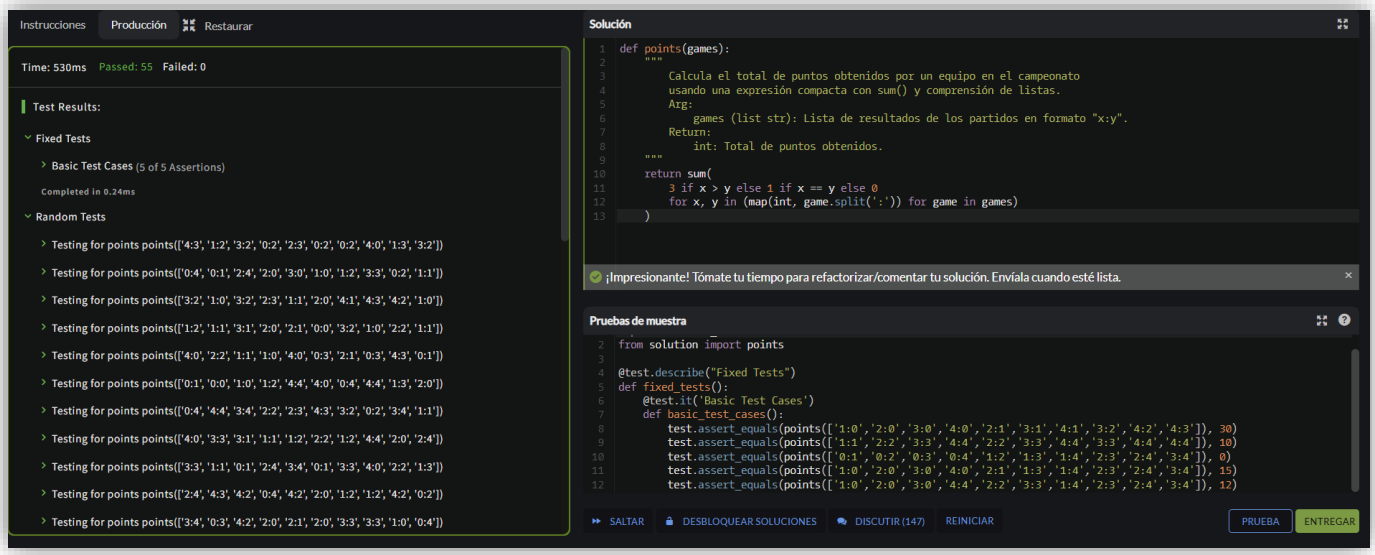
Necesitamos escribir una función que tome esta colección y devuelva el número de puntos que nuestro equipo (x) obtuvo en el campeonato según las reglas dadas anteriormente.

Notas:

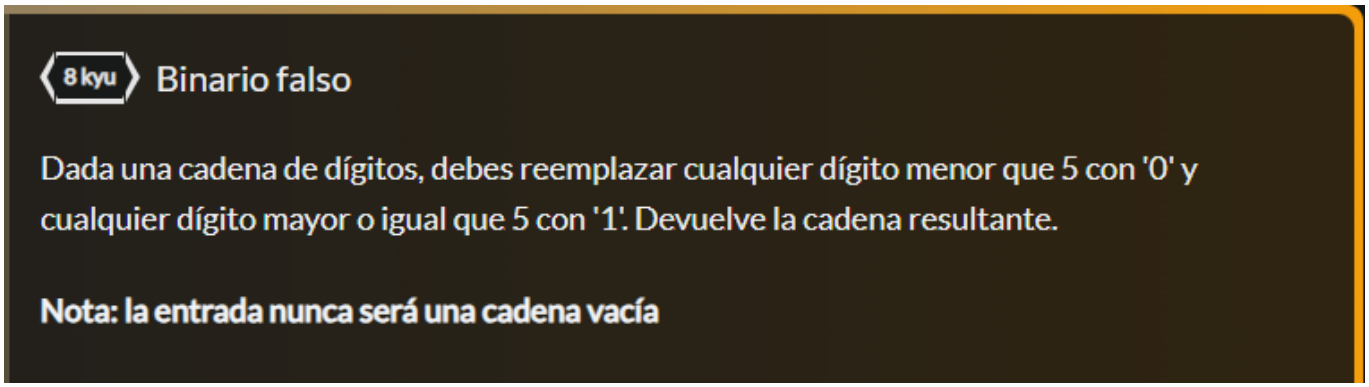
- Nuestro equipo siempre juega 10 partidos en el campeonato.
- 0 <= x <= 4
- 0 <= y <= 4

Solución

```python
def points(games):
    """
        Calcula el total de puntos obtenidos por un equipo en el campeonato
        usando una expresión compacta con sum() y comprensión de listas.
        Arg:
            games (list str): Lista de resultados de los partidos en formato "x:y".
        Return:
            int: Total de puntos obtenidos.
    """
    return sum(
        3 if x > y else 1 if x == y else 0
        for x, y in (map(int, game.split(':')) for game in games)
    )
```



KATA_14



Solución

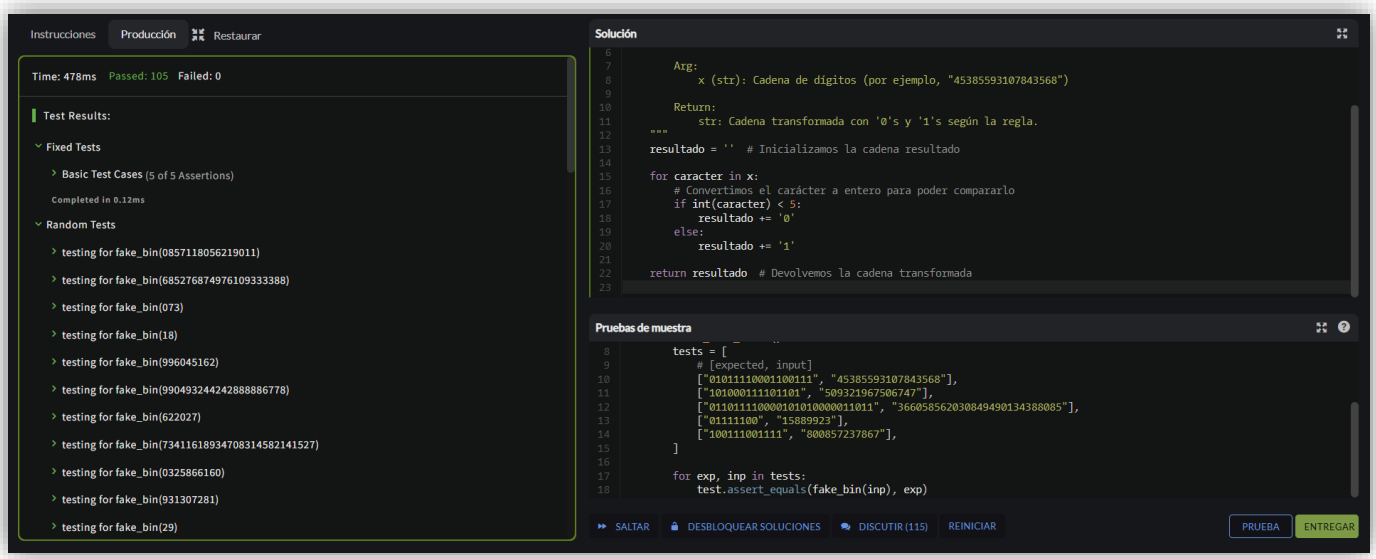
```python
def fake_bin(x):
    """
        Reemplaza cada dígito en la cadena:
            - Si el dígito es menor que 5 → se convierte en '0'
            - Si el dígito es 5 o mayor → se convierte en '1'

        Arg:
            x (str): Cadena de dígitos (por ejemplo, "45385593107843568")

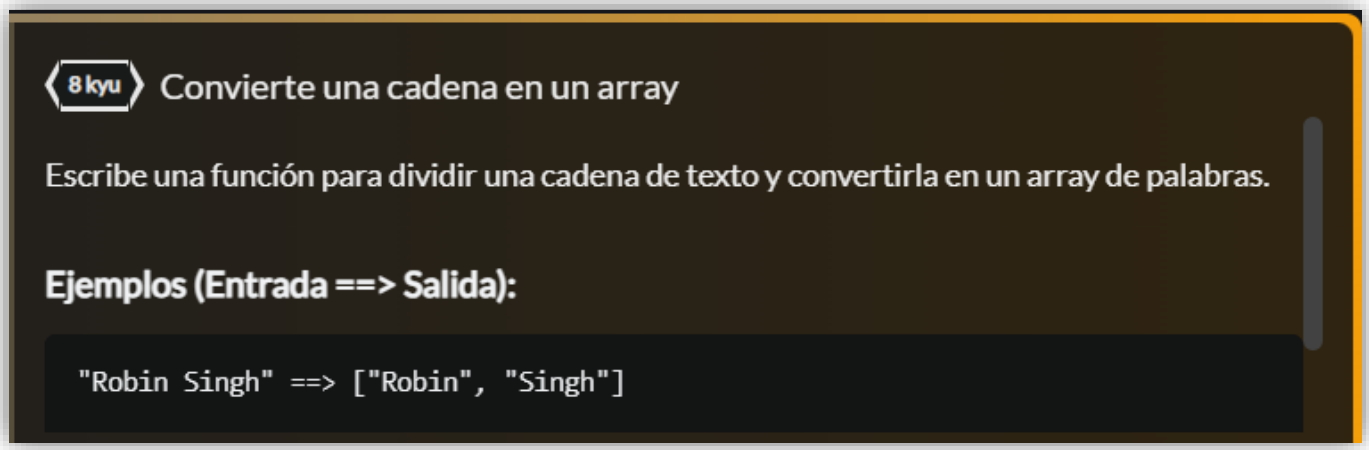
        Return:
            str: Cadena transformada con '0's y '1's según la regla.
    """
    resultado = ''  # Inicializamos la cadena resultado

    for caracter in x:
        # Convertimos el carácter a entero para poder compararlo
        if int(caracter) < 5:
            resultado += '0'
        else:
            resultado += '1'

    return resultado  # Devolvemos la cadena transformada
```



KATA_15



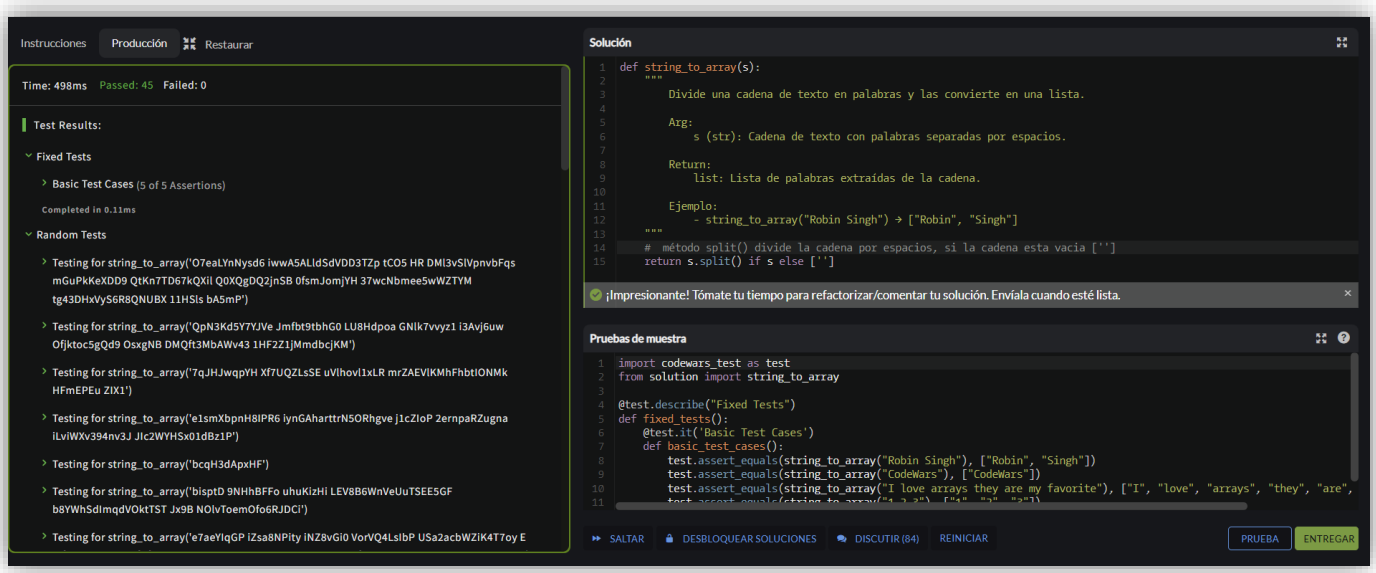
Solución

```
def string_to_array(s):
    """
        Divide una cadena de texto en palabras y las convierte en una lista.

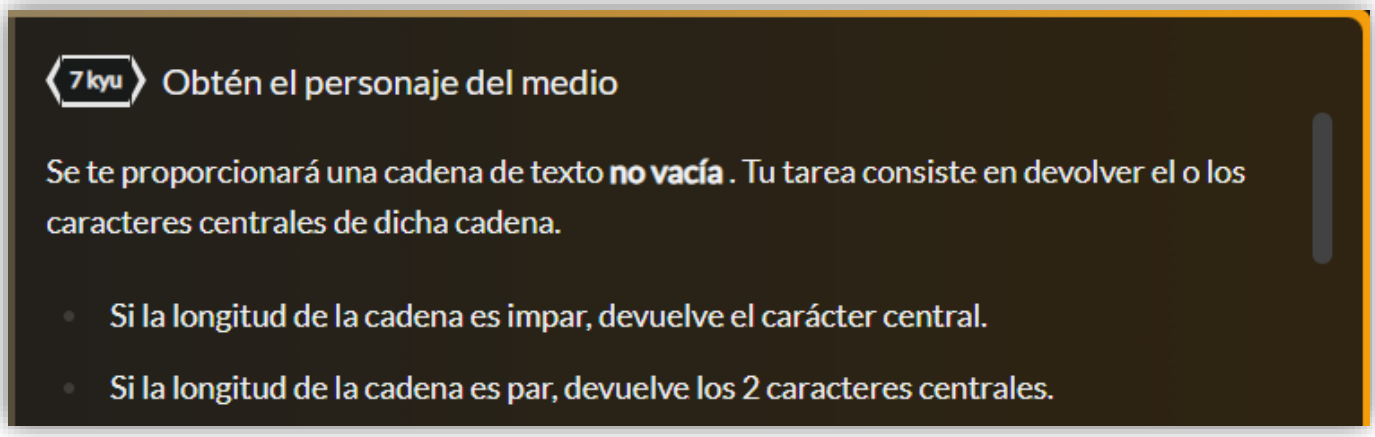
        Arg:
            s (str): Cadena de texto con palabras separadas por espacios.
```

```
    Return:
        list: Lista de palabras extraídas de la cadena.
        Si la cadena está vacía, devolvemos ['']

    Ejemplo:
        - string_to_array("Robin Singh") → ["Robin", "Singh"]
"""
#  método split() divide la cadena por espacios, si la cadena esta vacia devuelve ['']
return s.split() if s else ['']
```



KATA_16



Se te proporcionará una cadena de texto no vacía . Tu tarea consiste en devolver el o los caracteres centrales de dicha cadena.

Si la longitud de la cadena es impar, devuelve el carácter central.
Si la longitud de la cadena es par, devuelve los 2 caracteres centrales.
Ejemplos:

"test" --> "es"
"testing" --> "t"
"middle" --> "dd"
"A" --> "A"

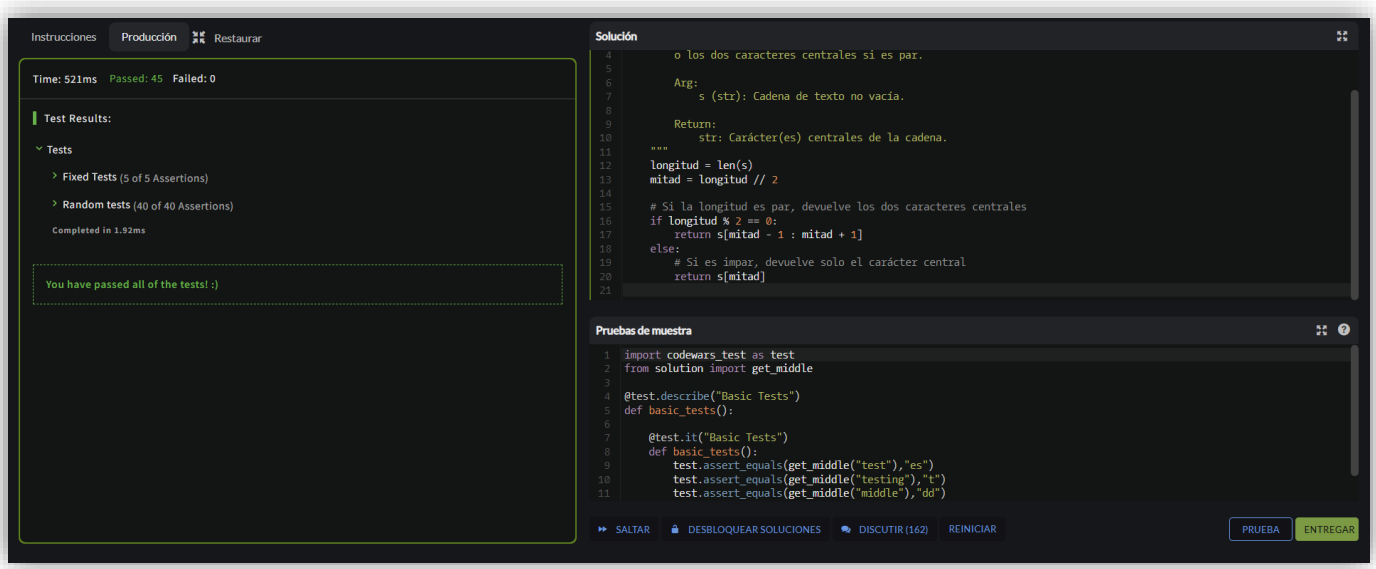
Solución

```python
def get_middle(s):
    """
    Devuelve el carácter central de una cadena si su longitud es impar,
    o los dos caracteres centrales si es par.

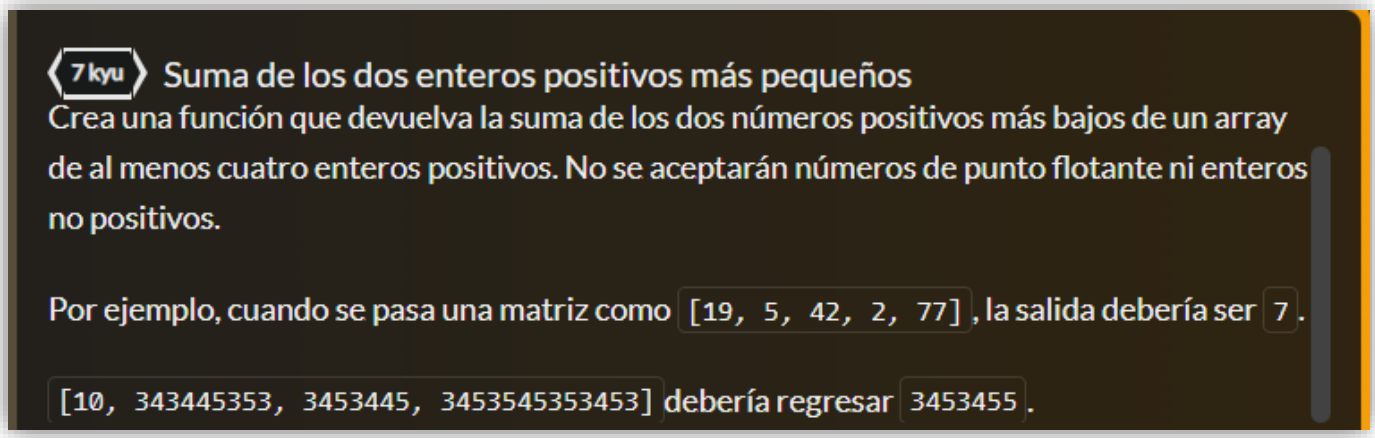
    Arg:
        s (str): Cadena de texto no vacía.

    Return:
        str: Carácter(es) centrales de la cadena.
    """
    longitud = len(s)
    mitad = longitud // 2
```

```python
# Si la longitud es par, devuelve los dos caracteres centrales
if longitud % 2 == 0:
    return s[mitad - 1 : mitad + 1]
else:
    # Si es impar, devuelve solo el carácter central
    return s[mitad]
```



KATA_17



7 kyu Suma de los dos enteros positivos más pequeños

Crea una función que devuelva la suma de los dos números positivos más bajos de un array de al menos cuatro enteros positivos. No se aceptarán números de punto flotante ni enteros no positivos.

Por ejemplo, cuando se pasa una matriz como `[19, 5, 42, 2, 77]`, la salida debería ser `7`.

`[10, 343445353, 3453445, 3453545353453]` debería regresar `3453455`.

Solución

```python
def sum_two_smallest_numbers(numbers):
    """
        Devuelve la suma de los dos números positivos más bajos de una lista.

        Restricciones:
        - La lista debe tener al menos 4 elementos.
        - Todos los elementos deben ser enteros positivos (> 0).
        - No se aceptan flotantes, ceros ni negativos.

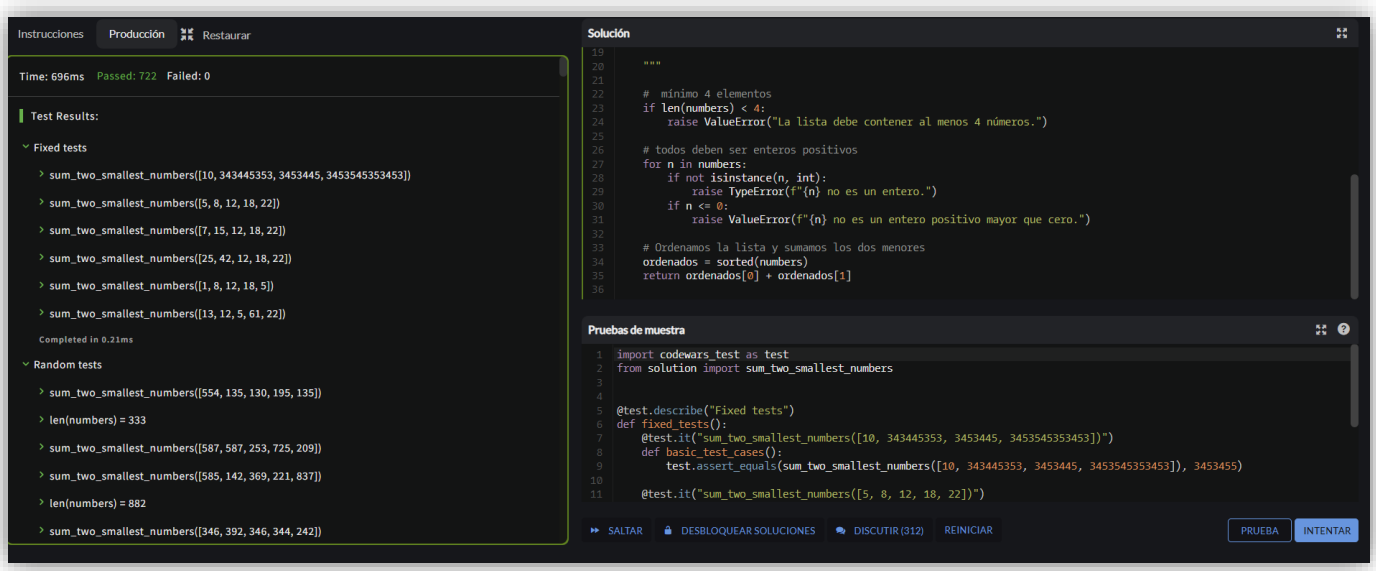
    Arg:
        numbers (list): Lista de enteros positivos.

    Return:
        int: Suma de los dos menores números positivos.
    Raises:
        ValueError: Si la lista tiene menos de 4 elementos o contiene números no válidos.
        TypeError: Si algún elemento no es un entero.
    """

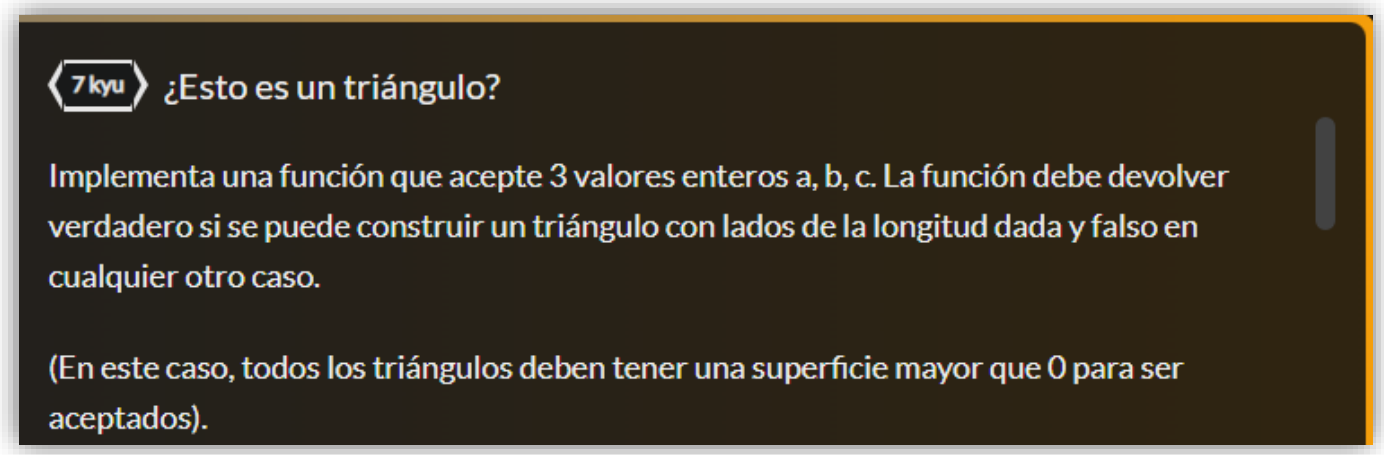
    #  mínimo 4 elementos
    if len(numbers) < 4:
        raise ValueError("La lista debe contener al menos 4 números.")
```

```python
# todos deben ser enteros positivos
for n in numbers:
    if not isinstance(n, int):
        raise TypeError(f"{n} no es un entero.")
    if n <= 0:
        raise ValueError(f"{n} no es un entero positivo mayor que cero.")

# Ordenamos la lista y sumamos los dos menores
ordenados = sorted(numbers)
return ordenados[0] + ordenados[1]
```



KATA_18



Ejemplos:

Input -> Output
1,2,2 -> true
4,2,3 -> true
2,2,2 -> true
1,2,3 -> false
-5,1,3 -> false
0,2,3 -> false
1,2,9 -> false

Para determinar si tres lados pueden formar un triángulo se basa en la desigualdad triangular, una propiedad fundamental de la geometría euclidiana.

Para que tres segmentos de longitud a, by $c$formen un triángulo válido, deben cumplir:

$$a + b > c, a + c > b, b + c > a$$

Estas tres condiciones garantizan que **cada par de lados juntos sea mayor que el tercer lado**, lo cual es necesario para que los extremos puedan encontrarse y formar una figura cerrada.

Además, los lados deben ser **mayores que cero**:

$$a > 0, b > 0, c > 0$$

Esto asegura que el triángulo tenga una **superficie mayor que cero**. Si algún lado es cero o negativo, no hay figura geométrica válida.

Solución

```python
def is_triangle(a, b, c):
    """
    Determina si tres lados pueden formar un triángulo válido.

    Requisitos:
        - Los tres valores deben ser enteros positivos (> 0).
        - Deben cumplir la desigualdad triangular:
            a + b > c, a + c > b, b + c > a

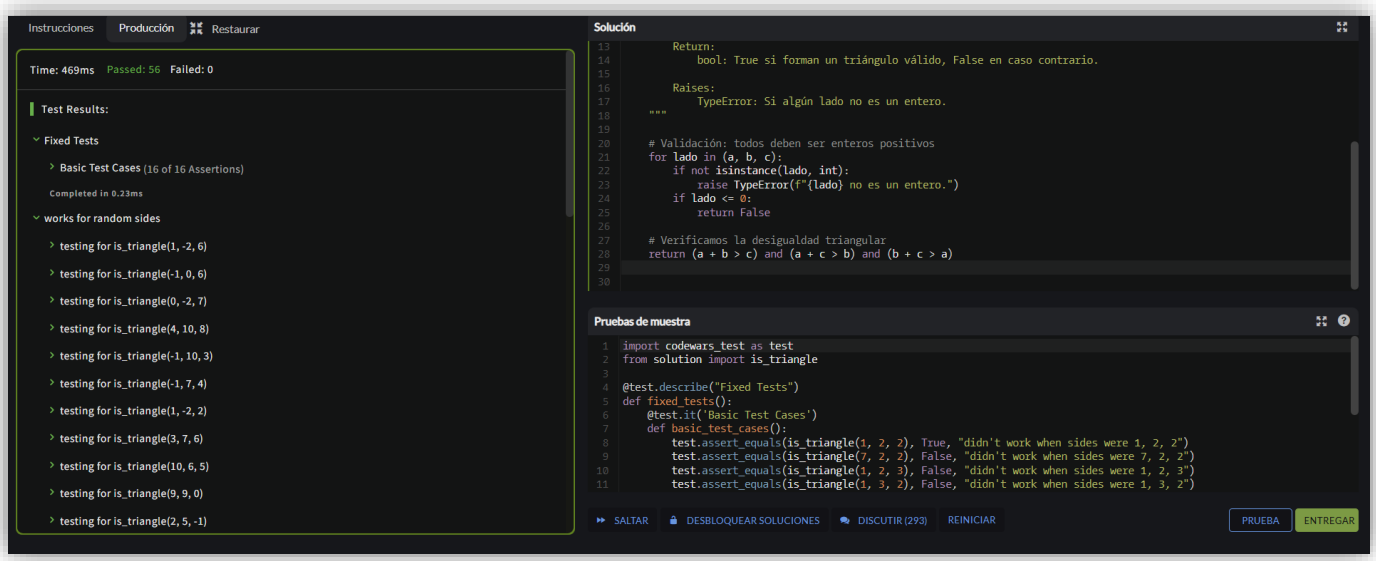
    Arg:
        a, b, c (int): Longitudes de los lados.

    Return:
        bool: True si forman un triángulo válido, False en caso contrario.

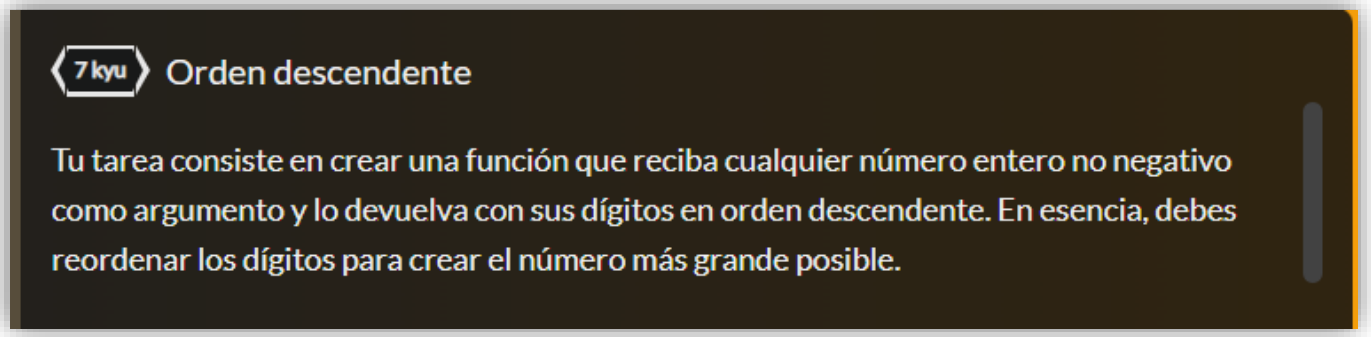
    Raises:
        TypeError: Si algún lado no es un entero.
    """

    # Validación: todos deben ser enteros positivos
    for lado in (a, b, c):
        if not isinstance(lado, int):
            raise TypeError(f"{lado} no es un entero.")
        if lado <= 0:
            return False

    # Verificamos la desigualdad triangular
    return (a + b > c) and (a + c > b) and (b + c > a)
```



KATA_19



Solución

```
def descending_order(num):
    """
    Recibe un número entero no negativo y devuelve el número formado
    por sus dígitos en orden descendente.

    Restricciones:
      - El número debe ser un entero.
      - Debe ser no negativo (≥ 0).
```

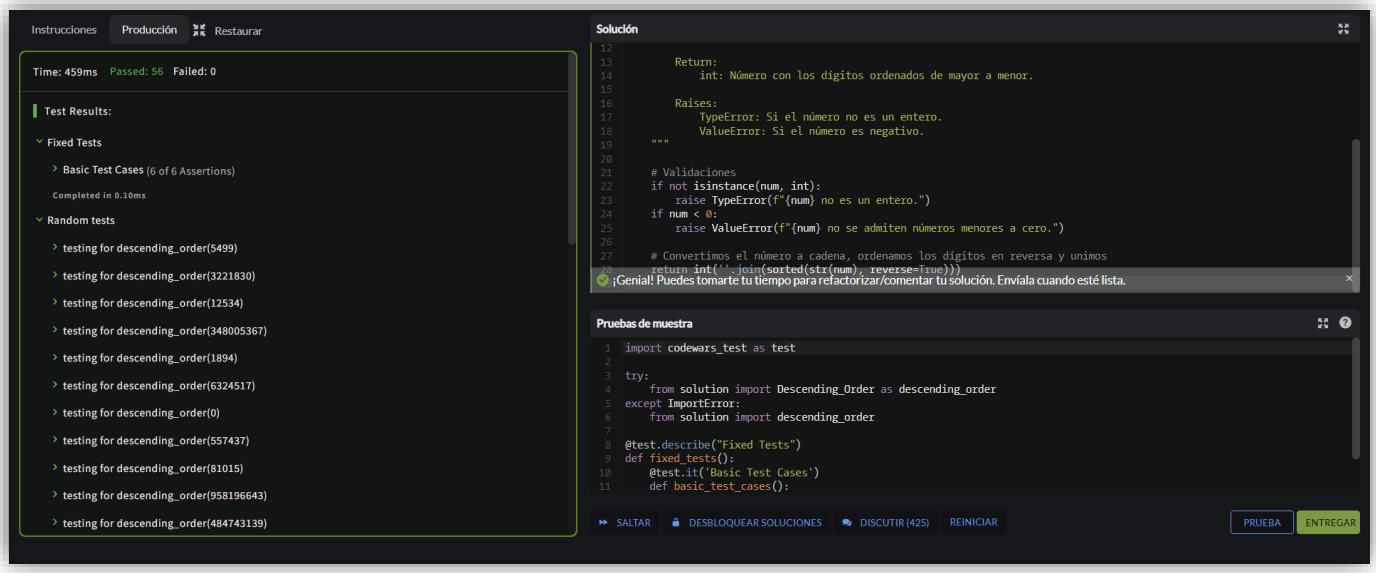
```
    Arg:
        num (int): Número entero no negativo.

    Return:
        int: Número con los dígitos ordenados de mayor a menor.

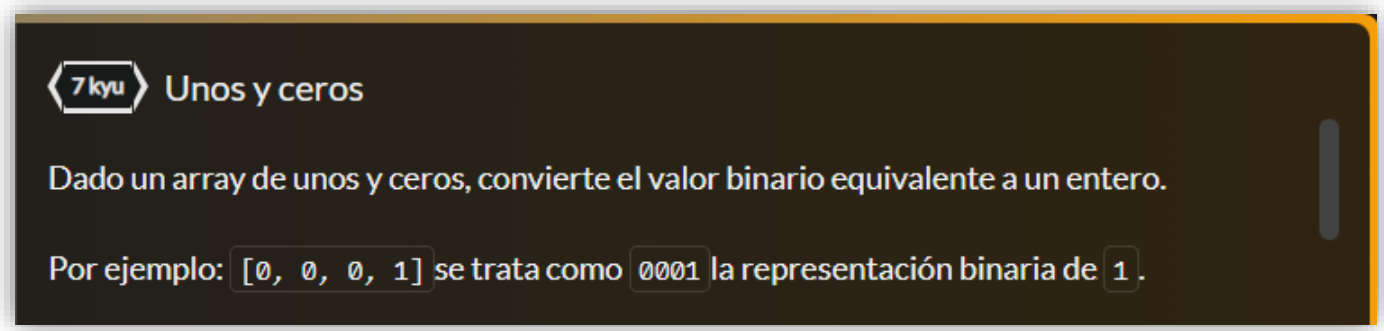
    Raises:
        TypeError: Si el número no es un entero.
        ValueError: Si el número es negativo.
    """

    # Validaciones
    if not isinstance(num, int):
        raise TypeError(f"{num} no es un entero.")
    if num < 0:
        raise ValueError(f"{num} no se admiten números menores a cero.")

    # Convertimos el número a cadena, ordenamos los dígitos en reversa y unimos
    return int(''.join(sorted(str(num), reverse=True)))
```



KATA_20



El sistema binario es un sistema de numeración en base 2. Solo utiliza dos dígitos: **0** y **1**. Cada posición representa una potencia de 2, empezando desde la derecha:

$$\text{Valor} = b_n \cdot 2^n + b_{n-1} \cdot 2^{n-1} + \cdots + b_1 \cdot 2^1 + b_0 \cdot 2^0$$

Donde $b_i \in \{0,1\}$es el valor del bit en la posición i.

Esta lista representa el número binario **1011**, que se interpreta así:

$$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11$$

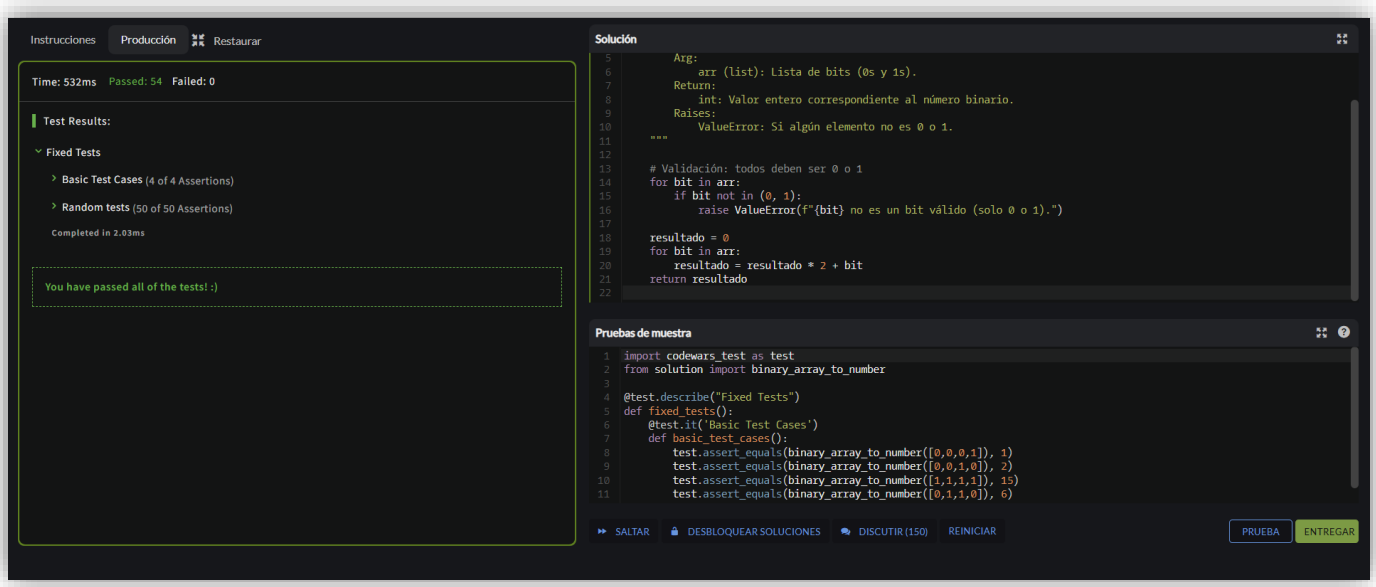
Solución

```python
def binary_array_to_number(arr):
    """
    Convierte un array de unos y ceros en su valor entero equivalente.

    Arg:
        arr (list): Lista de bits (0s y 1s).
    Return:
        int: Valor entero correspondiente al número binario.
    Raises:
        ValueError: Si algún elemento no es 0 o 1.
    """

    # Validación: todos deben ser 0 o 1
    for bit in arr:
        if bit not in (0, 1):
            raise ValueError(f"{bit} no es un bit válido (solo 0 o 1).")
```

```
resultado = 0
for bit in arr:
    resultado = resultado * 2 + bit
return resultado
```



BIBLIOGRAFIA

- Apuntes del Profesor.
- Python para todos. Explorando la información con Python 3. Charles R. Severance
- https://docs.python.org/es/3/library/copy.html
- https://www.w3schools.com/python/
- https://www.codewars.com/